

---

---

# Recurrent Neural Networks for NLP

— Intro to AI - Spring 2025 Class —

---

---

Eleni Metheniti - [eleni.metheniti@univ-tlse3.fr](mailto:eleni.metheniti@univ-tlse3.fr)

# Table of contents

1. [What is a neural network?](#)
2. [Recurrent Neural Networks](#)
  1. [Introduction to RNNs](#)
  2. [RNNs & NLP](#)
3. [Long Short-Term Memory NNs](#)
4. [Encoder-Decoder architecture](#)
5. [Attention Mechanism](#)
6. [Practical session: Training an RNN](#)

# 1. What is a neural network?

# Neural Networks

- Machine learning **model**
- Learning **features** directly from data
- Types of neural networks
  - Convolutional Neural Networks (CNNs) → great for image processing
  - Recurrent Neural Networks (RNNs, LSTMs) → great for sequential data
  - Transformers → great for... everything!

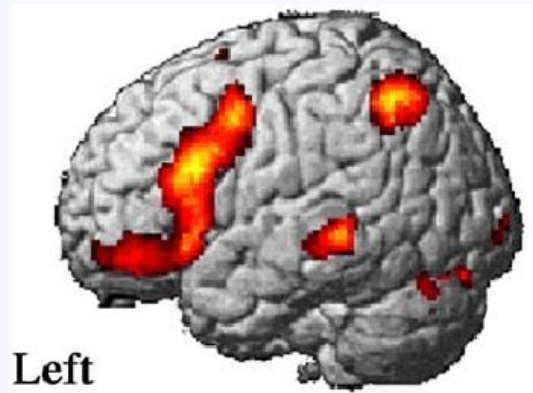
# Neural Networks: Neuron

- **Neuron**: the building block of a neural network  
(other names: **node**, **cell**)
- Stores information, is connected to other neurons



# Neural Networks: Neuron

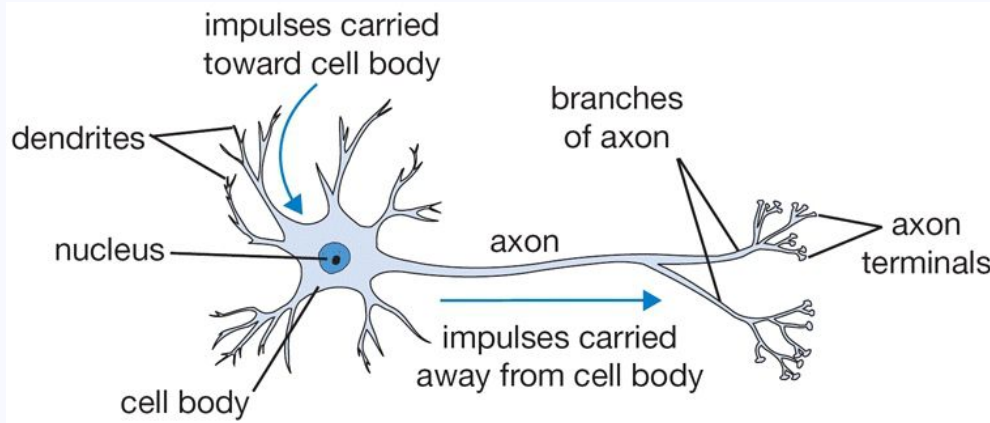
- **Neuron**: the building block of a neural network (other names: **node**, **cell**)
- Stores information, is connected to other neurons



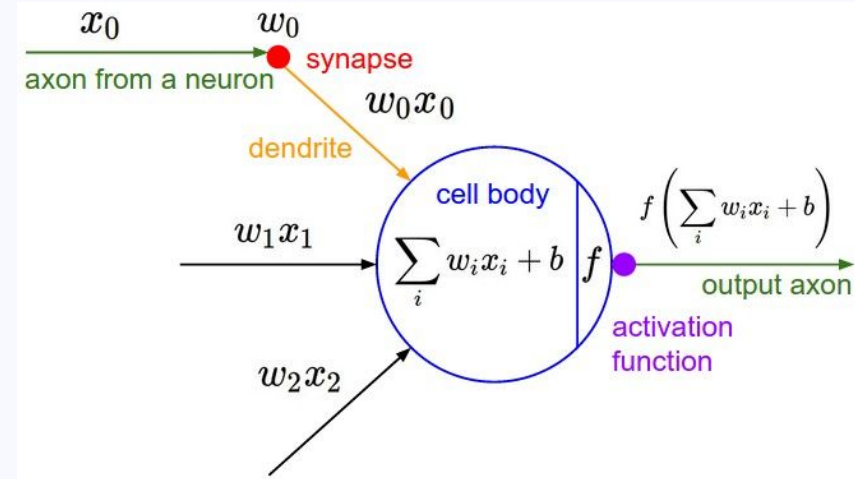
Yokoyama, Satoru, et al. "Cortical activation in the processing of passive sentences in L1 and L2: An fMRI study." *Neuroimage* 30.2 (2006): 570-579.



# Neural Networks: Neuron



Biological neuron



Artificial neuron

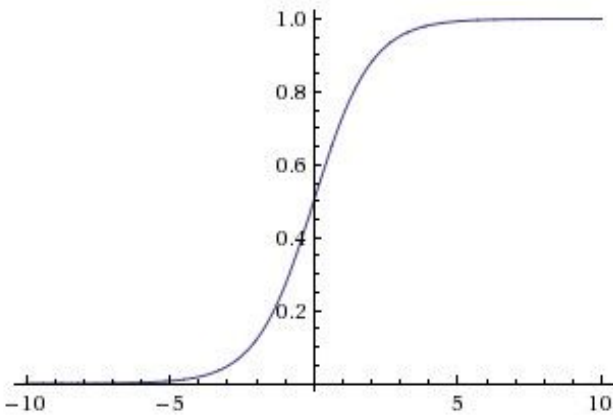
# Neural Networks: Neuron

Inside a neuron...

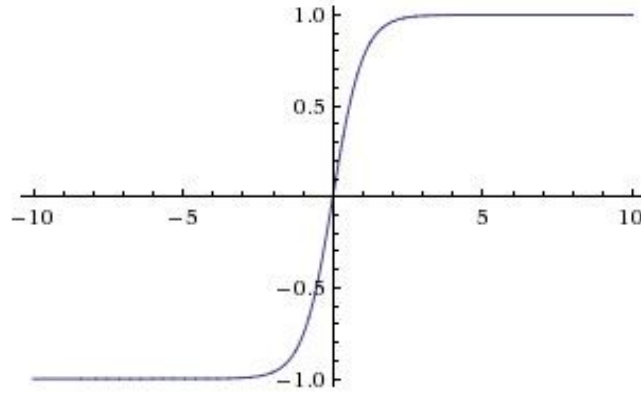
- Dot product of **input** and **weights**: the connection between two neurons
- Add the **bias**: a value that also influences the importance of the neuron
- Apply **activation function**: the *firing rate* of the neuron



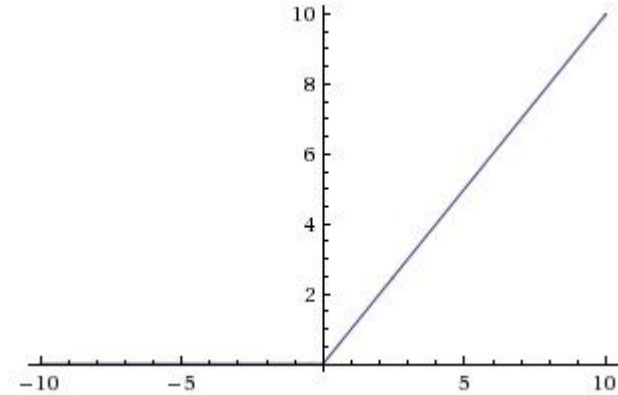
# Neural Networks: Activation functions



sigmoid (0 - 1)



tanh (-1 - 1)



ReLU (0 - z)

Run an activation function:

<https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions>

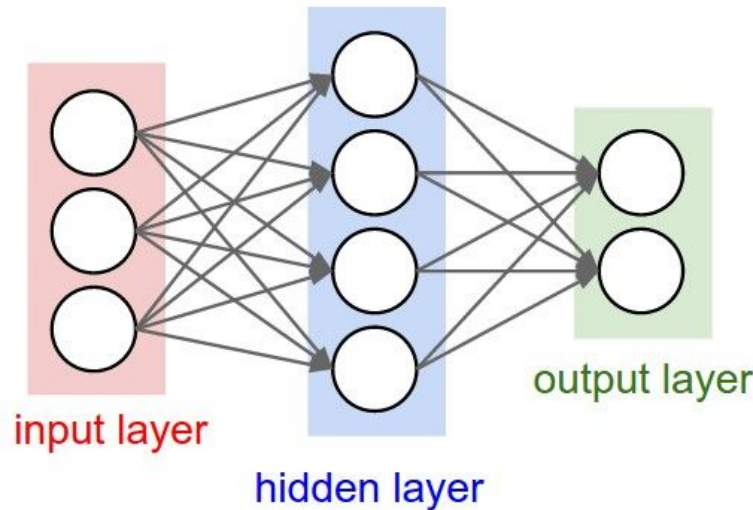
See more activation functions:

<https://cs231n.github.io/neural-networks-1>  
<https://sebastianraschka.com/faq/docs/activation-functions.html>

# Neural Networks: Layer

- **Layer:** a collection of neurons
- **The outputs of neurons become inputs to other neurons**

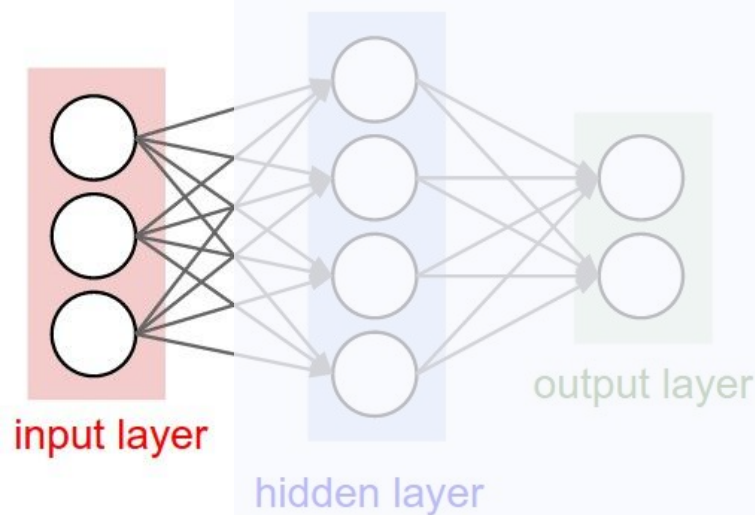
neurons of the same layer are **not** connected to each other...



...but each neuron is connected to **every** neuron of the next layer = fully-connected layer

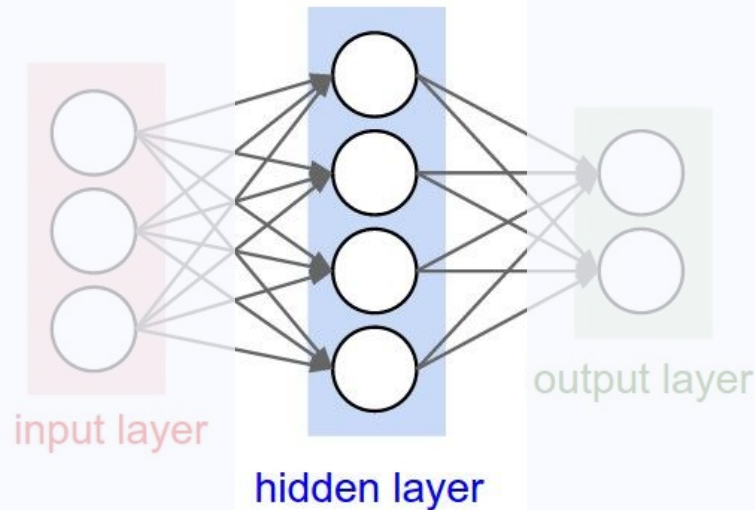
# Neural Network: Layer

- **Input layer:** first layer of the NN, contains **encoded data**



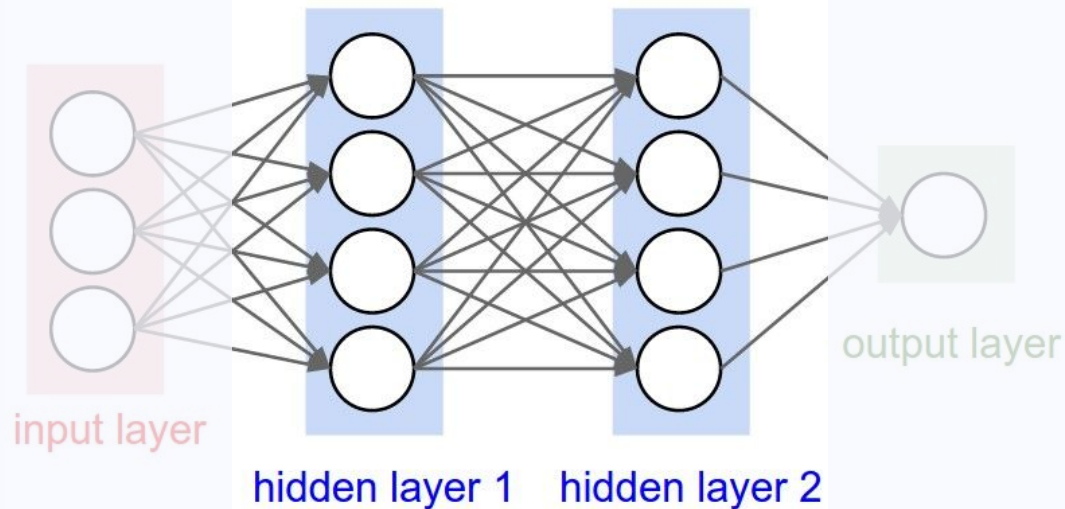
# Neural Network: Layer

- **Hidden layer(s):** learn different aspects about the data it received



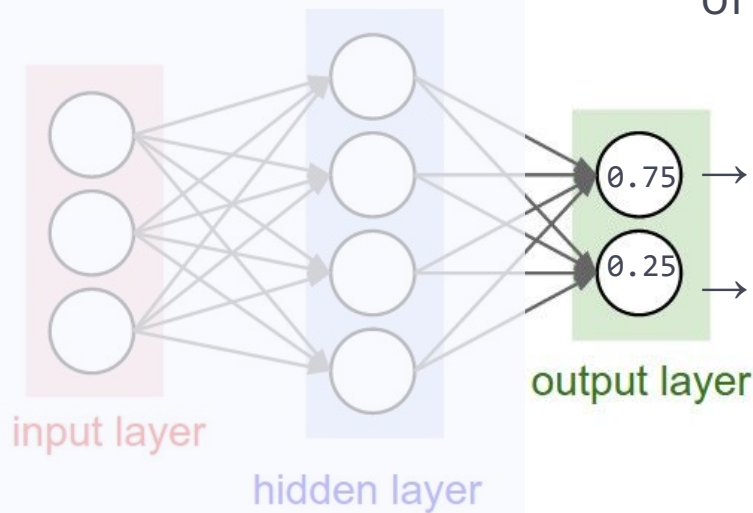
# Neural Network: Layer

- **Hidden layer(s):** learn different aspects about the data it received
- One or more hidden layers, interacting with the next!



# Neural Network: Layer

- **Output layer:** no function, contains values (probability of a token, label...)



e.g. binary classification of "I hate peaches." :

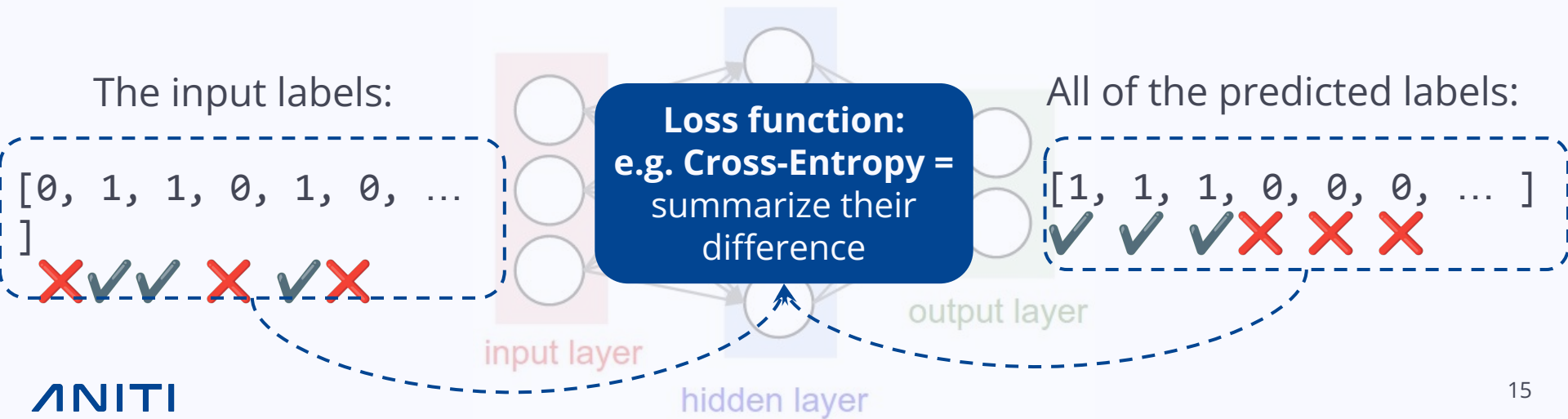
→ prob. of label 0 👎 = 75%

→ prob. of label 1 👍 = 25%

# Neural Network: Output

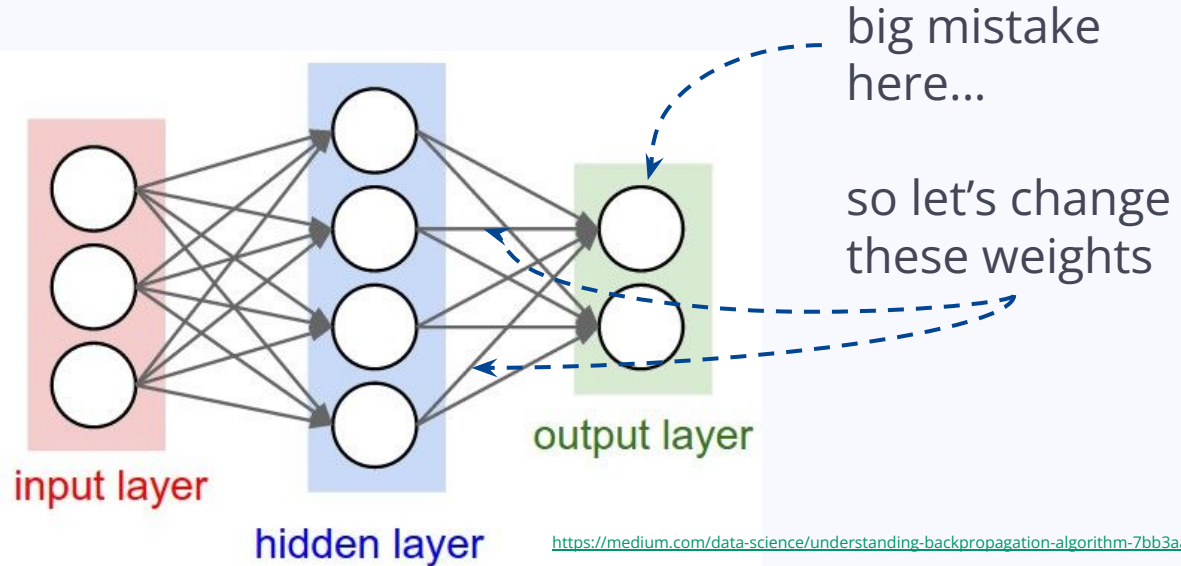
- **Output layer:** no function, contains values (probability of a token, label...)
- **Loss:** compare input values to predictions (= output values)

e.g. binary classification:



# Neural Network: Backpropagation

- **Loss:** compare input values to predictions (= output values)
- If it's bad... **backpropagation** will tell us who is responsible! 🕵️



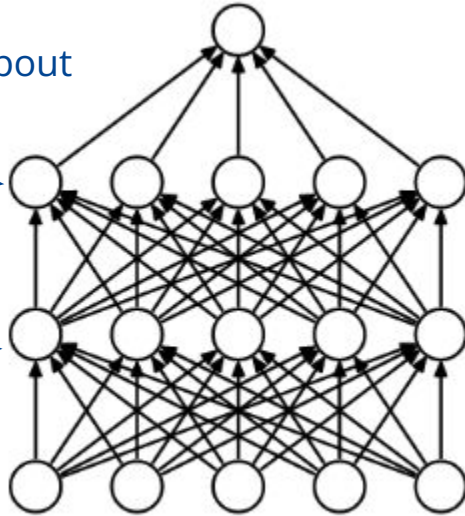


# Neural Network: Dropout rate

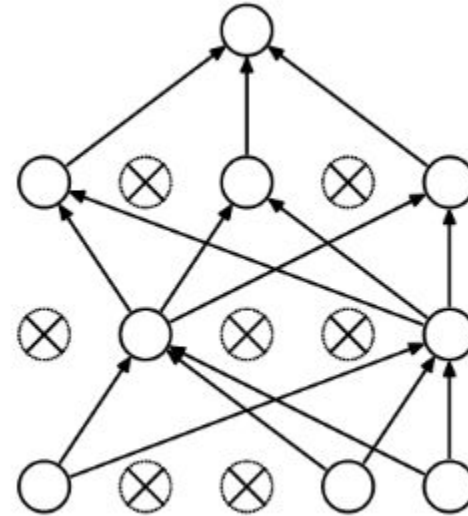
Temporarily “deactivate” some nodes of the input and hidden layer(s)

probability > dropout

probability < dropout



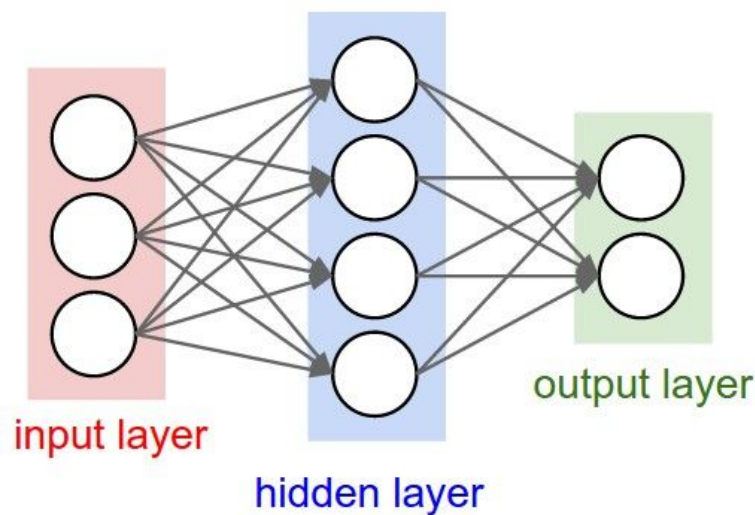
(a) Standard Neural Net



(b) After applying dropout.

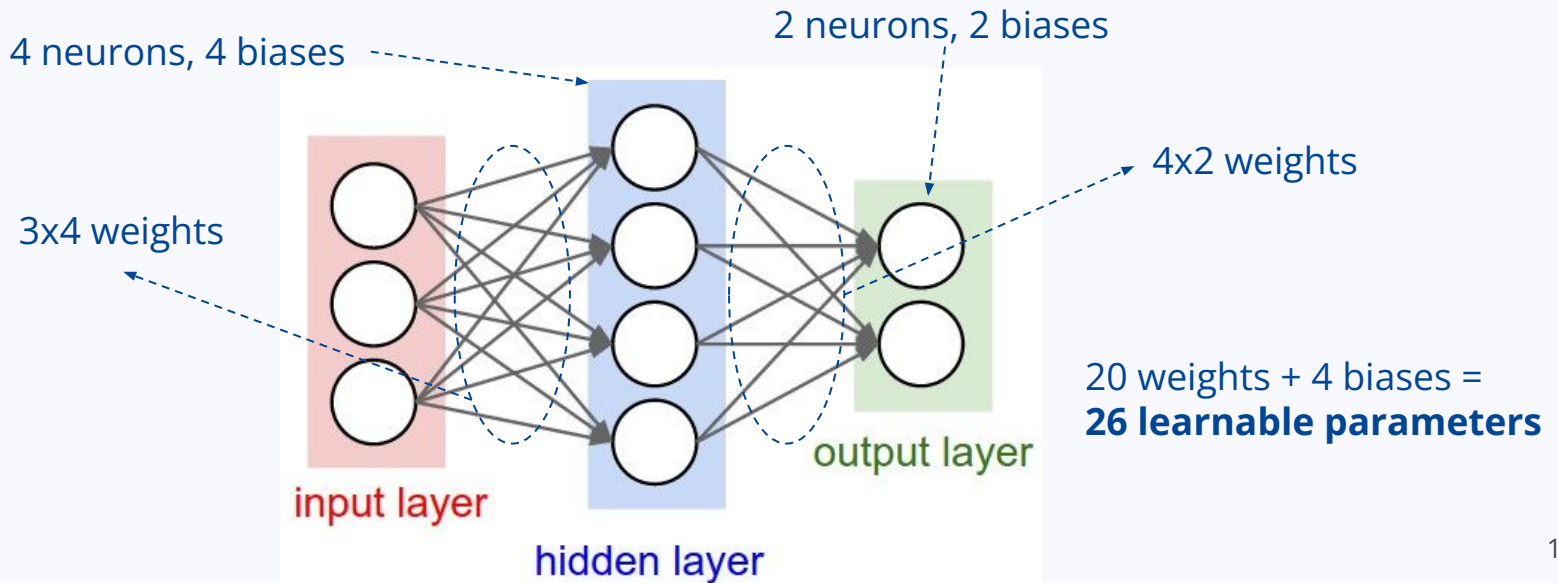
# Neural Network: Epoch

- **Loss:** compare input values to predictions (= output values)
- If it's bad...
- Solution: re-adjust weights & biases, redo it all in a new **epoch**!

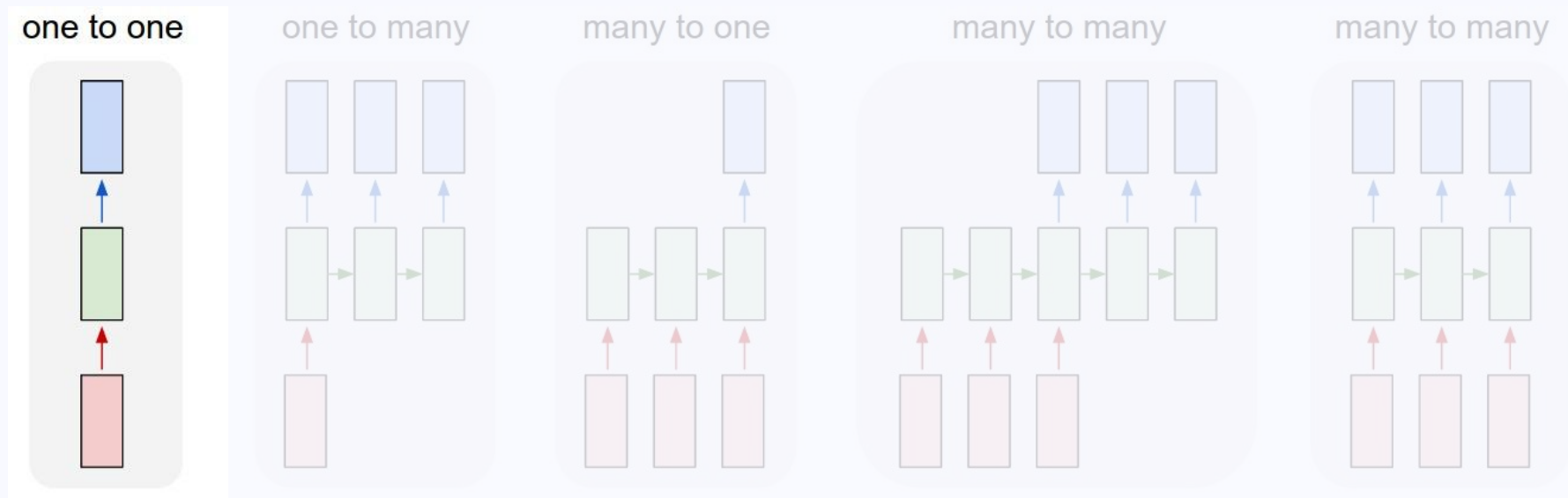


# Neural Network: Size

- Size based on **neurons** = Count of neurons of hidden layer(s) & output layer
- Size based on **learnable parameters** = Their biases + Their weights



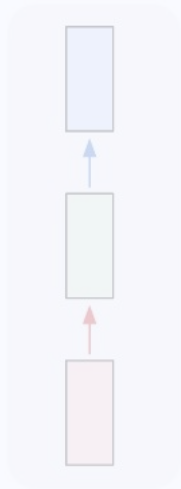
# Structure of a NN



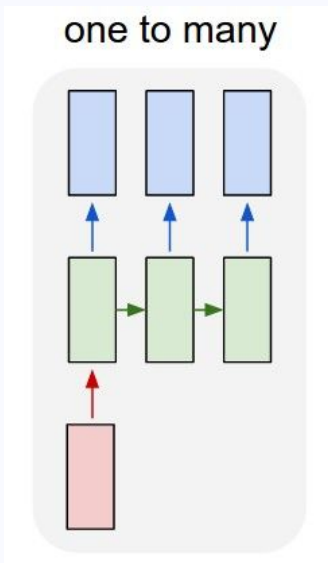
One fixed-size input → One fixed-size output (e.g. image classification)

# Structure of a NN

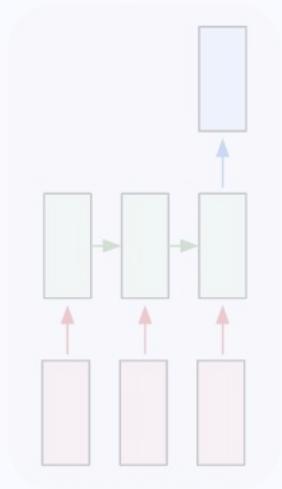
one to one



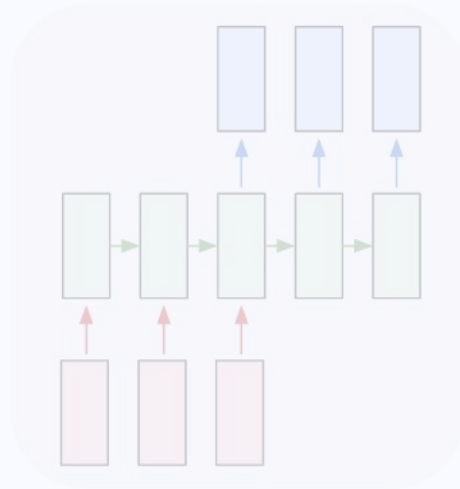
one to many



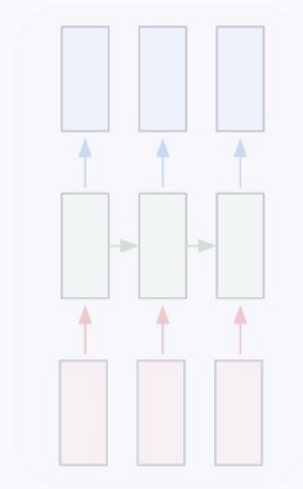
many to one



many to many



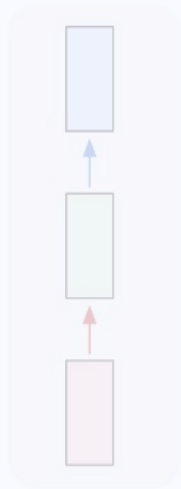
many to many



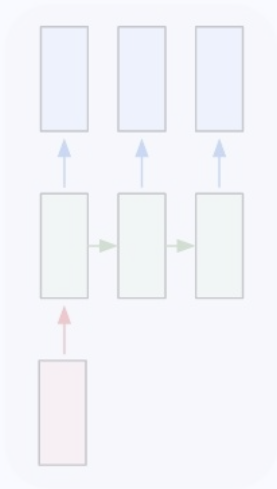
One fixed-size input → Output with multiple values (e.g. image captioning)

# Structure of a NN

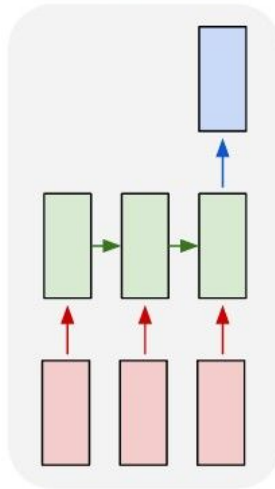
one to one



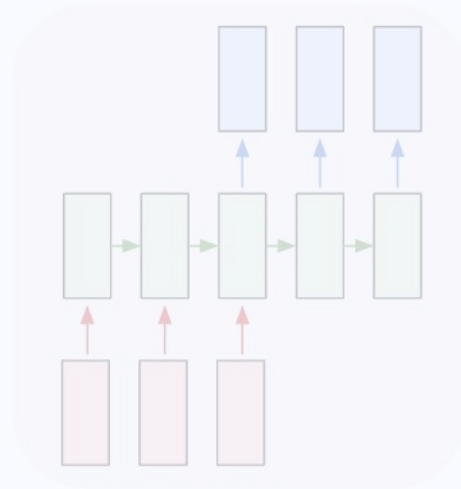
one to many



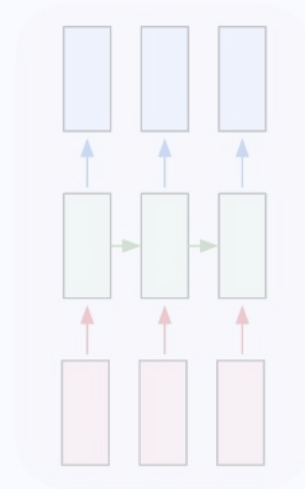
many to one



many to many



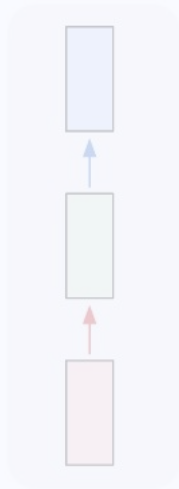
many to many



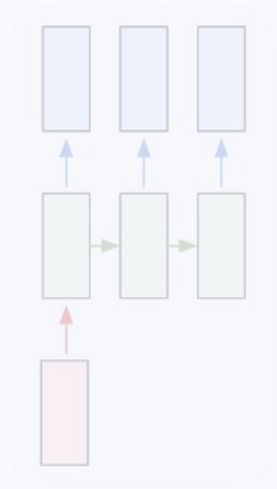
Input with multiple values → One fixed output (e.g. text classification)

# Structure of a NN

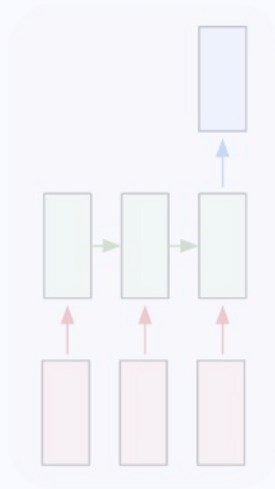
one to one



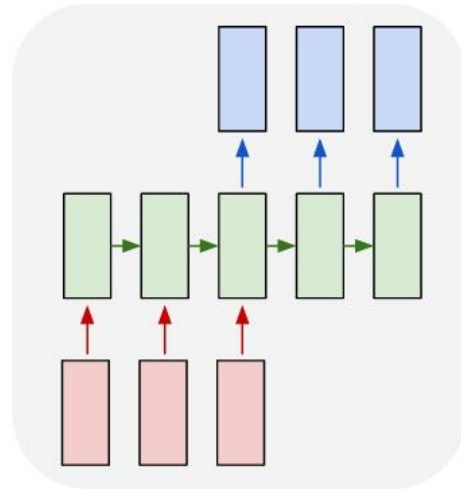
one to many



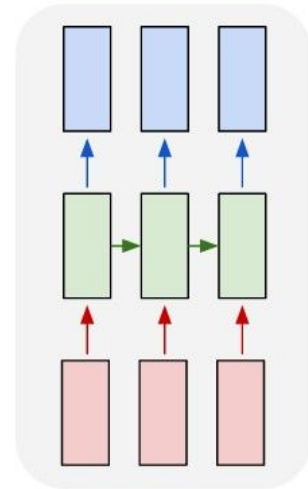
many to one



many to many



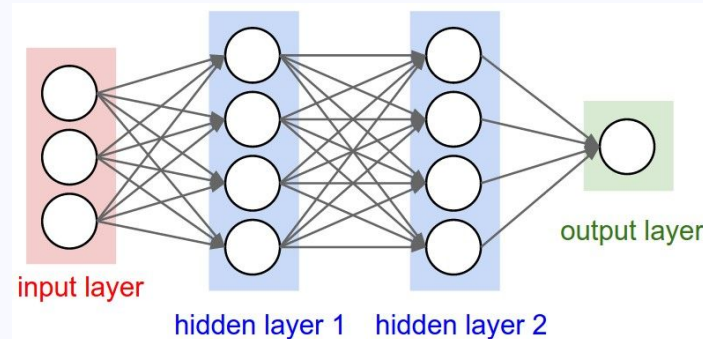
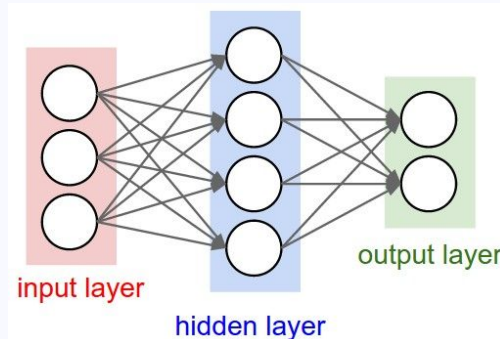
many to many



Input with multiple values → Output with multiple values (e.g. machine translation)

# Neural Network: Feedforward NN

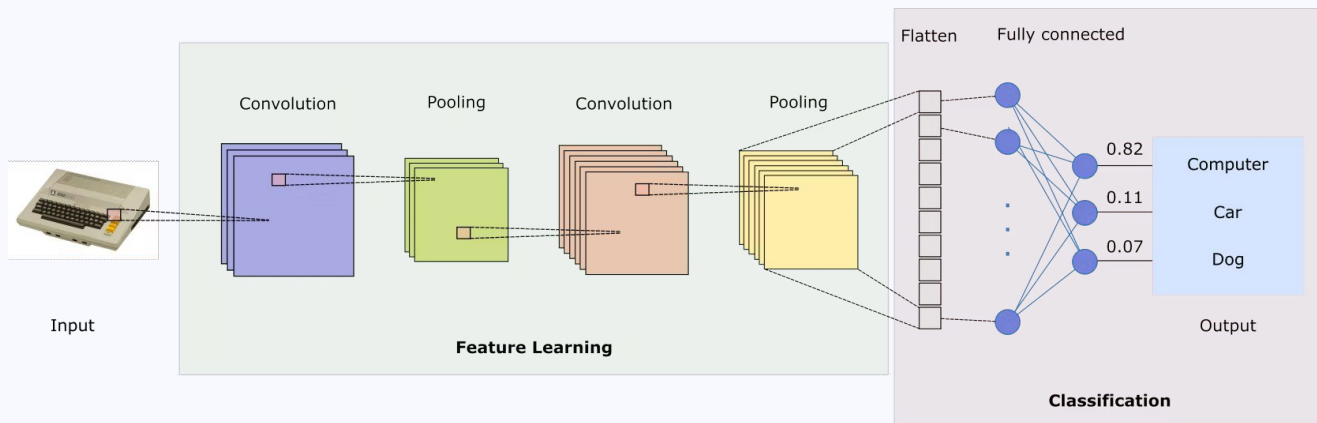
- Information flows in one direction
- (Some) drawbacks:
  - Information flows in one direction → limited contextual understanding!
  - Overfitting: good/bad with seen data, bad with unseen data
  - Requires feature engineering (see Lecture 5!)





# Convolutional Neural Networks

- **Feature maps** to detect features /patterns in the input data → NN
- (Some) drawbacks for NLP:
  - Fixed input size
  - Problem with long-distance relationships



<https://domino.ai/blog/gpu-accelerated-convolutional-neural-networks-with-pytorch>

# A Game of Sequential Data

- Recite the alphabet in your head...
  - In the right order!



# A Game of Sequential Data

- Recite the alphabet in your head...
  - In the right order! → A, B, C, D, E, F, G, H, ...
  - Backwards!



# A Game of Sequential Data

- Recite the alphabet in your head...
  - In the right order! → A, B, C, D, E, F, G, H, ...
  - Backwards! → Z, Y, X, W, V, U, ...
  - Starting from the letter “N” →



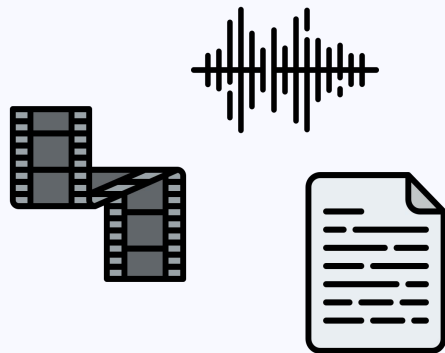
# A Game of Sequential Data

- Recite the alphabet in your head...
  - In the right order! → A, B, C, D, E, F, G, H, ...
  - Backwards! → Z, Y, X, W, V, U, ...
  - Starting from the letter “N” → N, O, P, Q, R, S, T, ...
- We have learned the alphabet as a **sequence**
- Difficult in other ways!



# Sequential data

- Video, audio, **text** are sequences
- But remember... 🍎🥧 ≠ 🍎📱
- We need a model that can:
  - handle context & position in the sentence
  - handle sentences with varying lengths
  - handle context left and right
  - handle long-distance relations



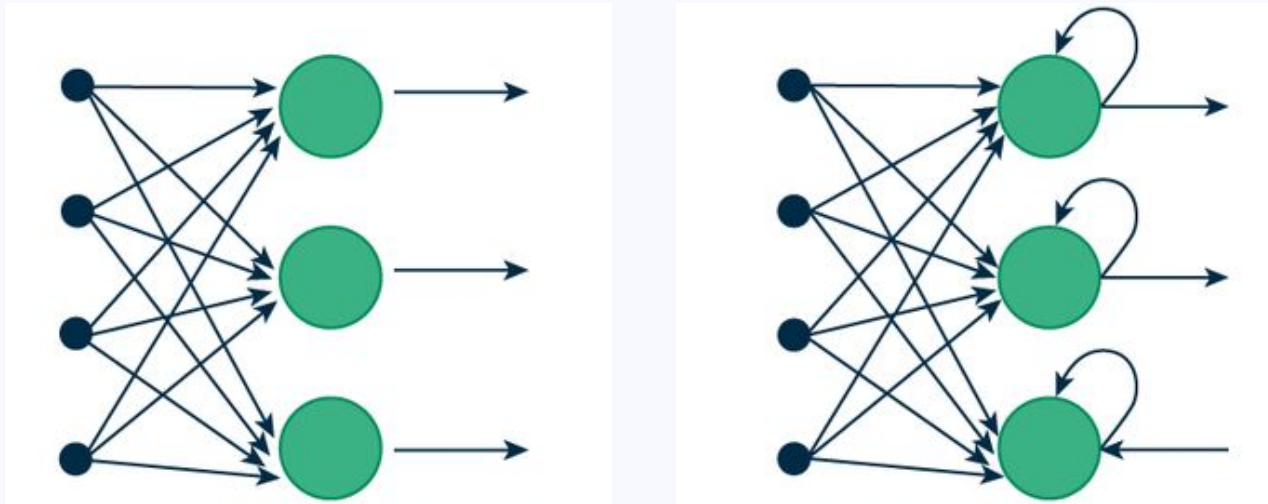
## 2. Recurrent Neural Networks

# Introduction to RNNs



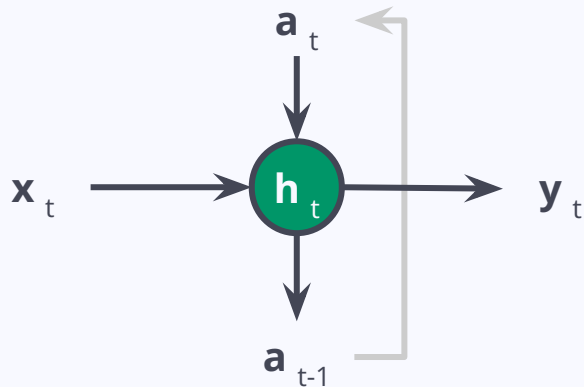
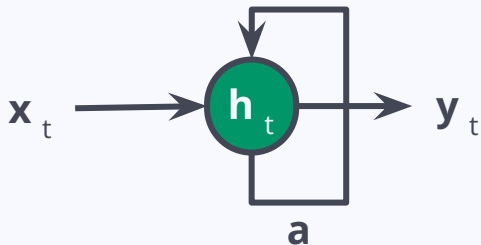
# What is an RNN?

- Like a feedforward NN... but better with **feedback!**
- **Loops** that allow information to go back into the network



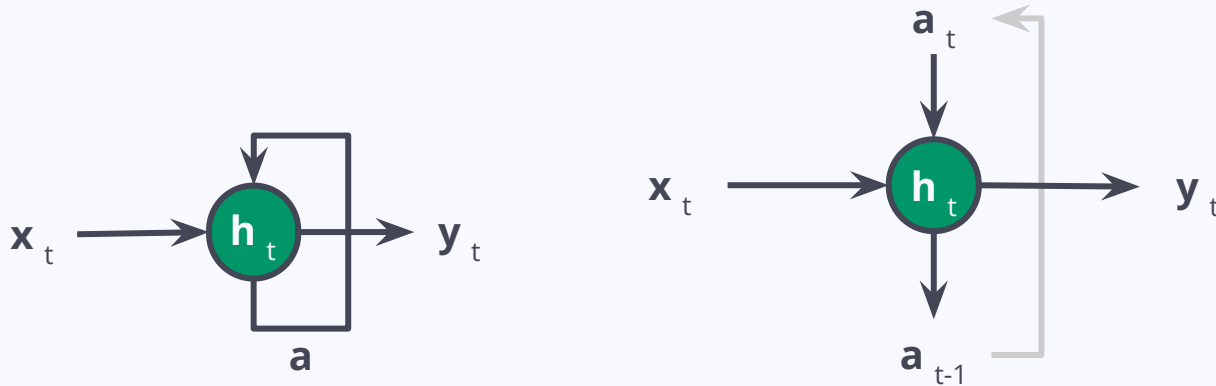
# Unfolding an RNN

- **Recurrent Unit:** Neuron that holds a **hidden state** that contains the past
  - $\mathbf{x}_t$  = **input** in *time t*
  - $\mathbf{y}_t$  = **output** in *time t*
  - $\mathbf{a}_{t-1}$  = **activation** in *time t-1* = the previous activation
  - $\mathbf{a}_t$  = **activation** in *time t* = the current activation
  - $\mathbf{h}_t$  = **hidden state** of the neuron in *time t*



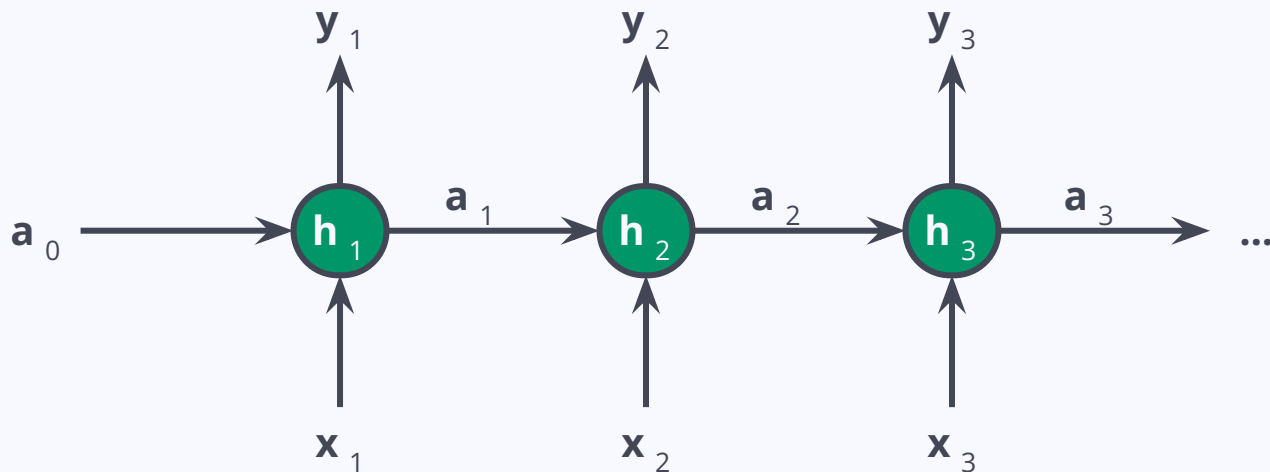
# Unfolding an RNN

- **Backpropagation Through Time (BPTT):** “learning from its errors”
- Current output = current input + memory of past inputs



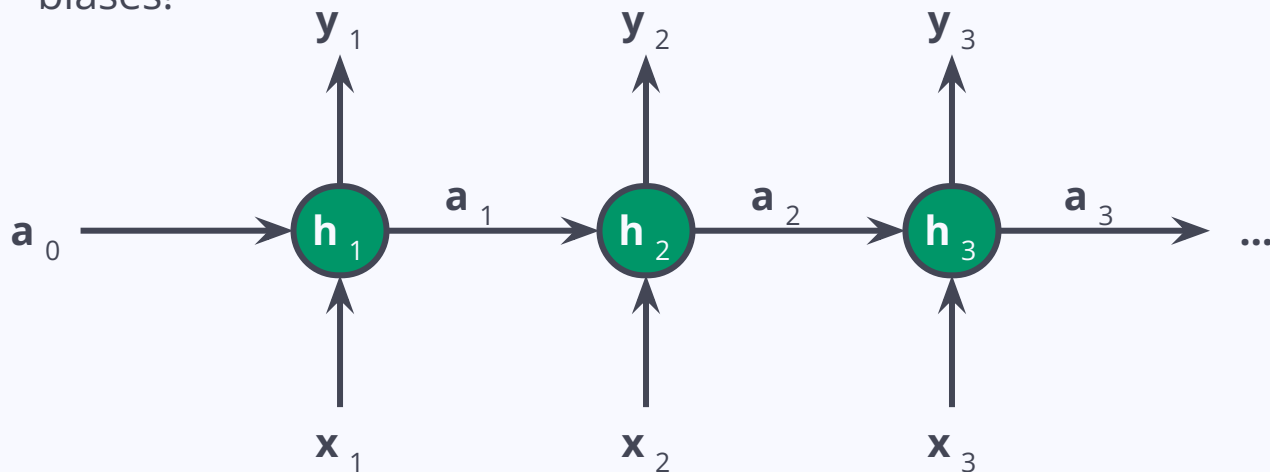
# Structure of an RNN

- **Hidden state ( $h_t$ ) calculation:**  $h_t = \sigma(U \cdot x_t + W \cdot h_{t-1} + B)$ 
  - $U, W$  = weights!
  - $B$  = biases!



# Structure of an RNN

- **Output ( $y_t$ ) calculation:**  $y_t = O(V \cdot h_t + C)$ 
  - $O$  = activation function
  - $V$  = weights!
  - $C$  = biases!



# RNNs: Applications

- Processing **sequential data**
  - Time-series: economics, etc.
  - NLP : words, sentences, etc. → Let's focus on this...

# RNNs & NLP

# NLP with RNNs: Example

**Binary Classification** of a sentence: "What time is it?"

-  → Category 0
-  → Category 1



# NLP with RNNs: Example

- Tokenization

What time is it?

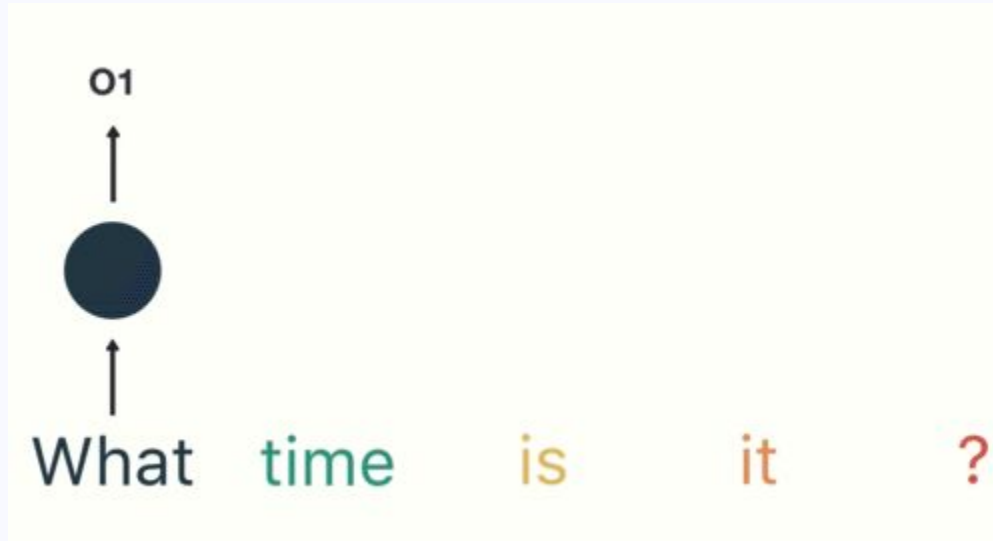
# NLP with RNNs: Example

- **1st timestep:** pass the word “What” to the model

What time is it ?

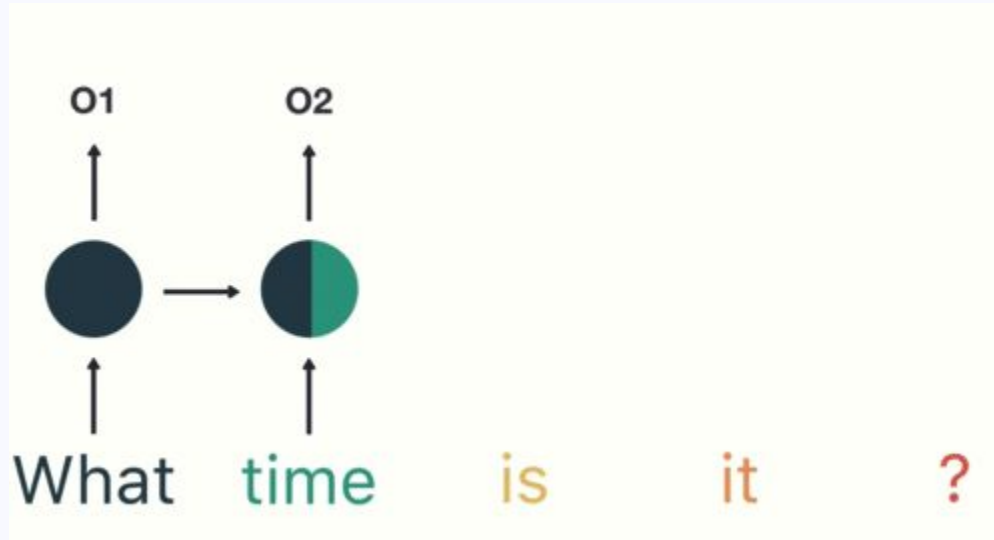
# NLP with RNNs: Example

- **2nd timestep:** pass the word “time” and the hidden state with “what”



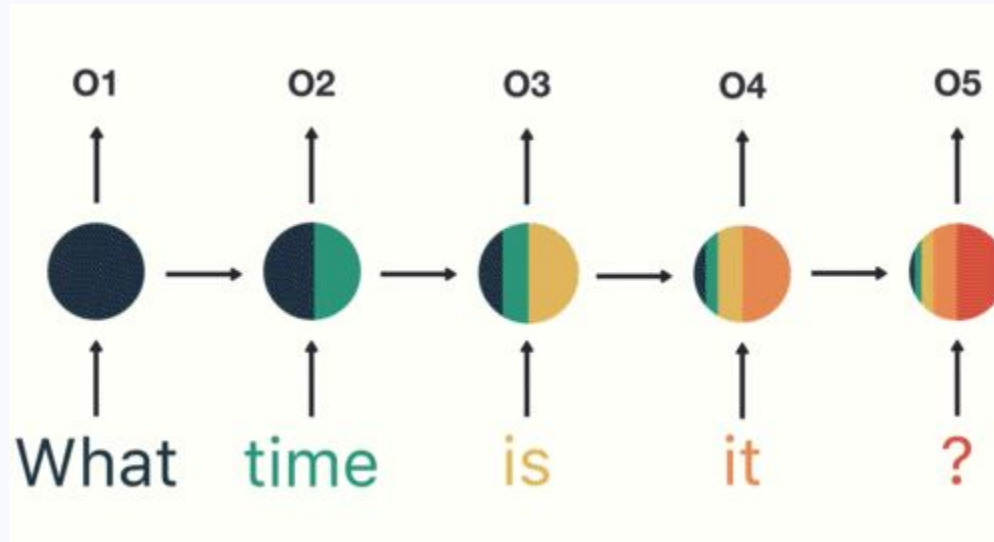
# NLP with RNNs: Example

- Repeat for all timesteps until end of sequence



# NLP with RNNs: Example

- Binary Classification:  $O_5$  = **Probabilities** of Category 0 🕒 & Category 1 ☀️



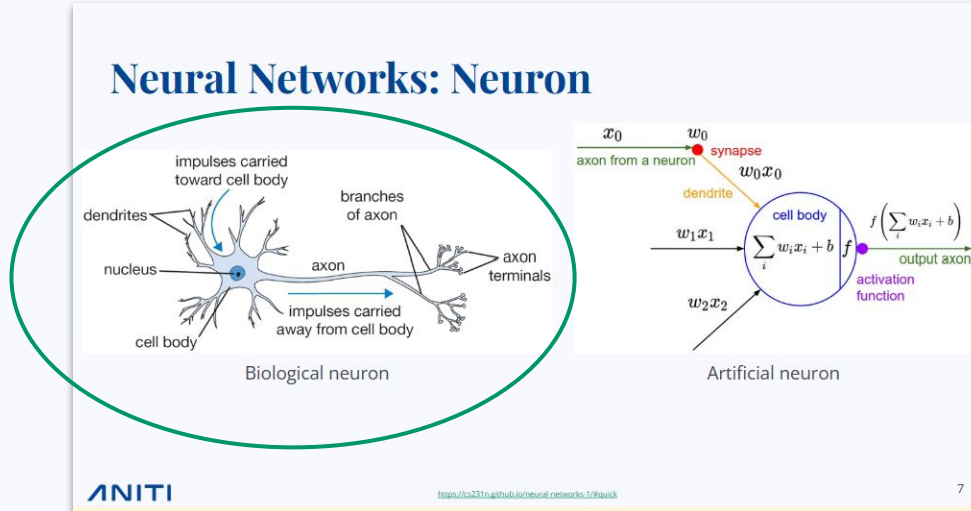
# Vanishing gradient problem

Do you remember what was in the first image in [Slide 7](#)?

# Vanishing gradient problem

Do you remember what was in the first image in [Slide 7](#)?

Human working memory has limits... and so do RNNs



# Vanishing gradient problem

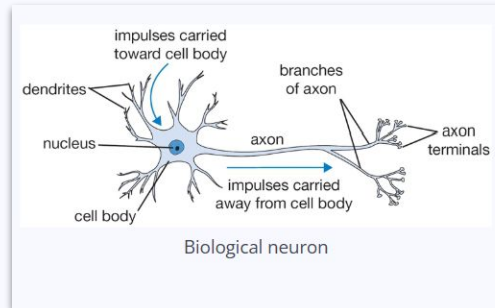
- **BPTT**: gradients exponentially shrink as timesteps pass
- Final timestep: early hidden states are “not important” anymore





# Exploding gradient problem

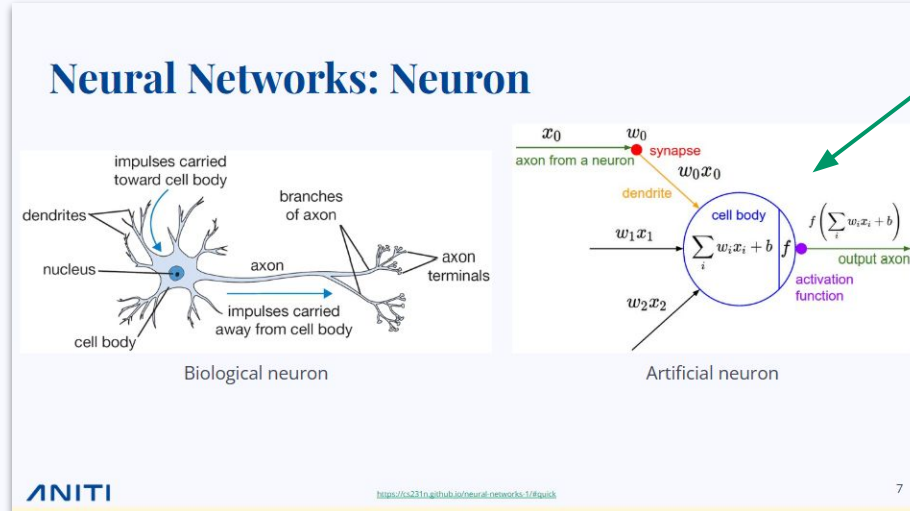
Now you remember what was in the first image in [Slide 7](#)!



# Exploding gradient problem

Now you remember what was in the first image in [Slide 7](#)!

...but it's not so important to our lecture today!



# Exploding gradient problem

- What if we change the weights to make some hidden states important...?
- “time” is now very important... but this could be a problem with more data!

$t_2$  = “what time”

$t_5$  = “what time is it ?”



# RNNs: Pros & Cons

- + Processing input of any length
- Computation being slow

# RNNs: Pros & Cons

- + Processing input of any length
- + Modeling using **past** samples, capture dependencies!
- Computation being slow
- Cannot use any **future** input!

# RNNs: Pros & Cons

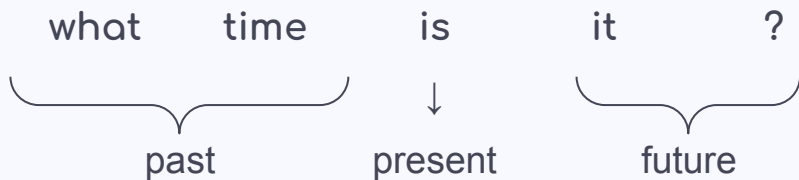
- + Processing input of any length
- + Modeling using **past** samples, capture dependencies!
- + Weights are shared across time
- Computation being slow
- Cannot use any **future** input!
- Vanishing/Exploding gradients: difficulty with **long sequences**

# RNNs: Pros & Cons

- + Processing input of any length
- + Modeling using **past** samples, capture dependencies!
- + Weights are shared across time
- Computation being slow
- Cannot use any **future** input!
- Vanishing/Exploding gradients: difficulty with **long sequences**

# Bidirectionality

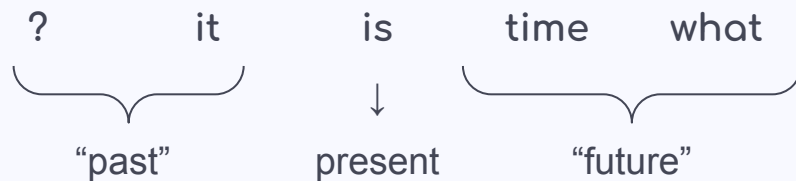
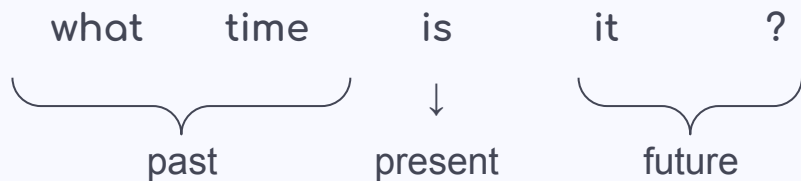
- NNs process input left → right:    past → present → ~~future~~
- But what if we **reversed the sequence**?



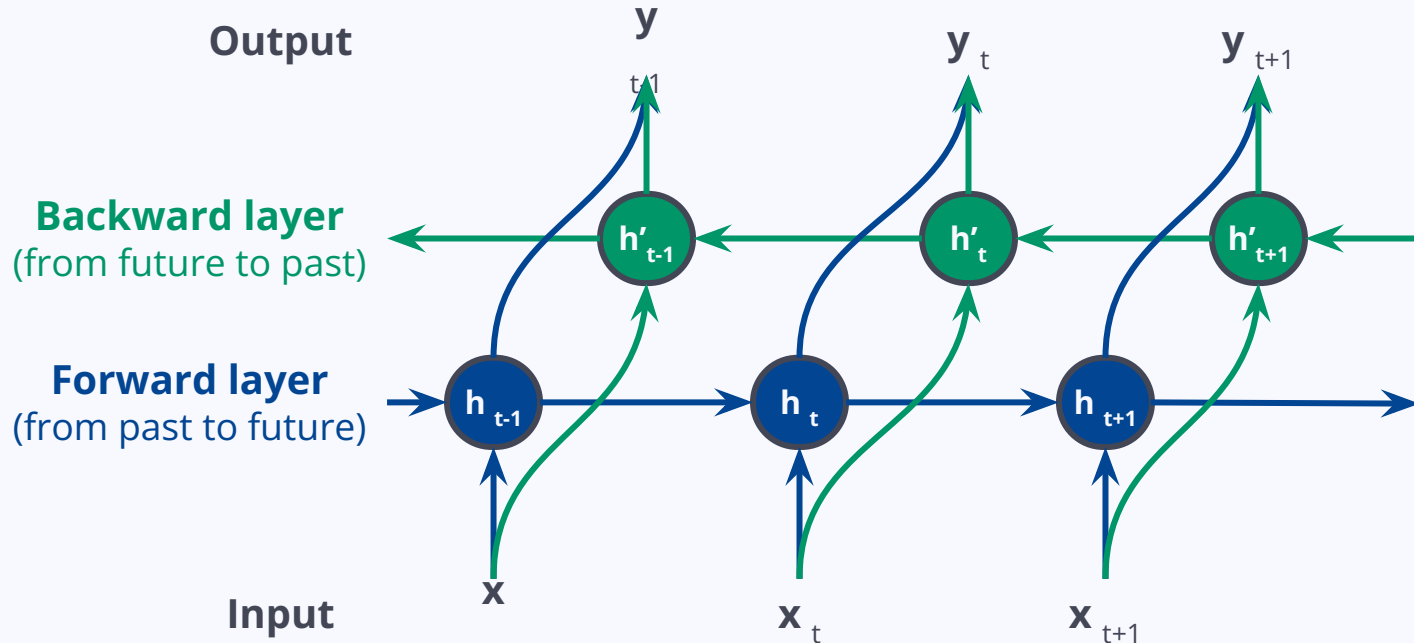


# Bidirectionality

- NNs process input left → right: past → present → ~~future~~
- But what if we **reversed the sequence**?
- The future is to the left → accessible to the model as a “past” state!

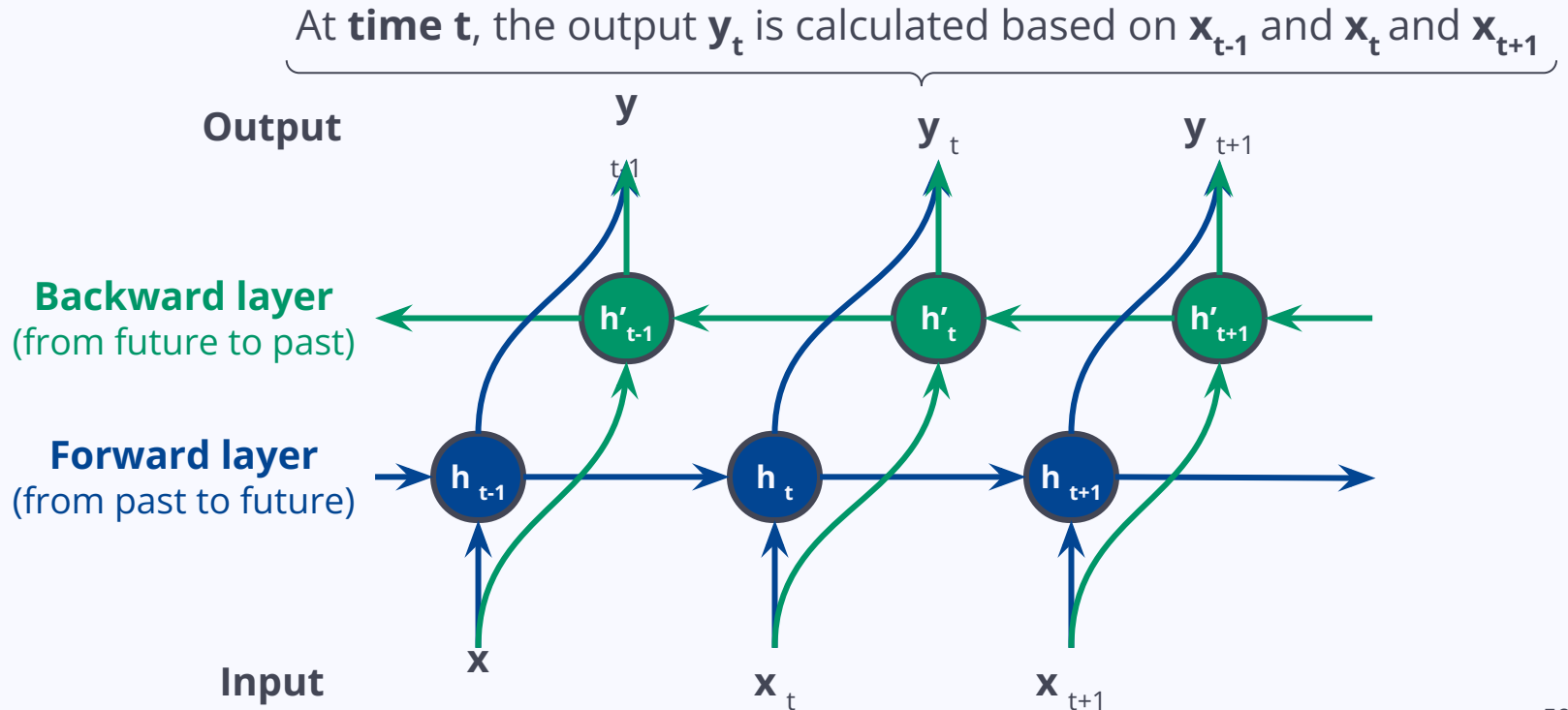


# Bidirectionality: bi-RNN



<https://www.geeksforgeeks.org/bidirectional-recurrent-neural-network/>

# Bidirectionality: bi-RNN



# RNNs: Pros & Cons

- + Processing input of any length
- + Modeling using **past** samples, capture dependencies!
- + Weights are shared across time

- Computation being slow
- Cannot use any **future** input!

Improve the RNN architecture!

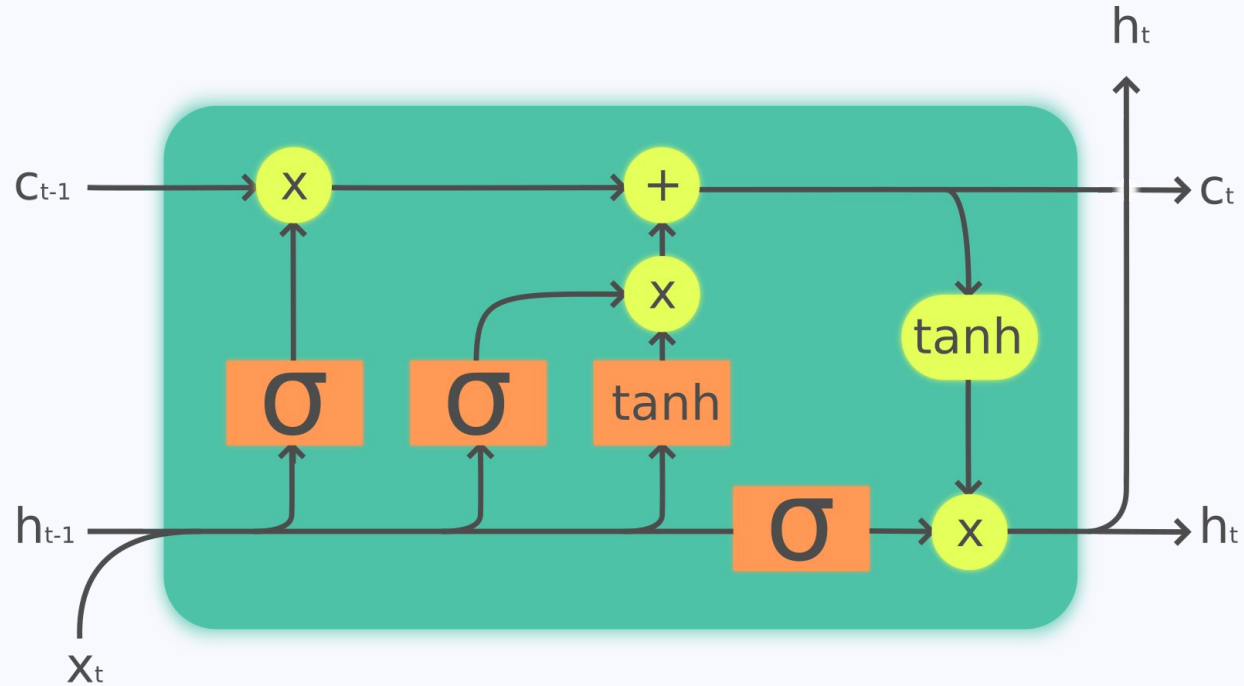
- Vanishing/Exploding gradients: difficulty with **long sequences**

# 3. Long Short-Term Memory Neural Networks

# What is an LSTM?

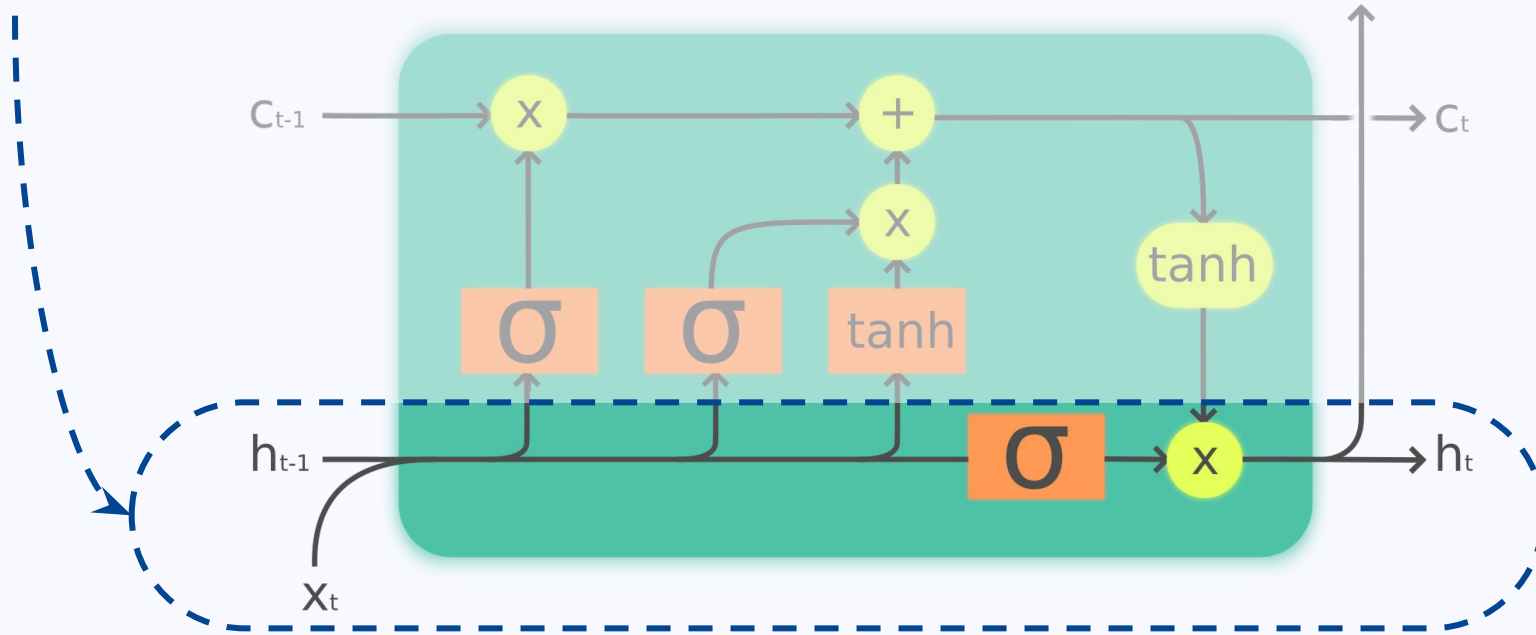
- **Long Short-Term Memory** Neural Network: improvement on RNNs
- Designed to avoid vanishing/exploding gradients
- RNNs: one linear path for all previous states
- LSTMs:
  - one path for **long-term “memories”** (states) → directly influence output
  - one path for **short-term “memories”** (states) → follow the flow of information

# LSTM: Cell



# LSTM: Cell

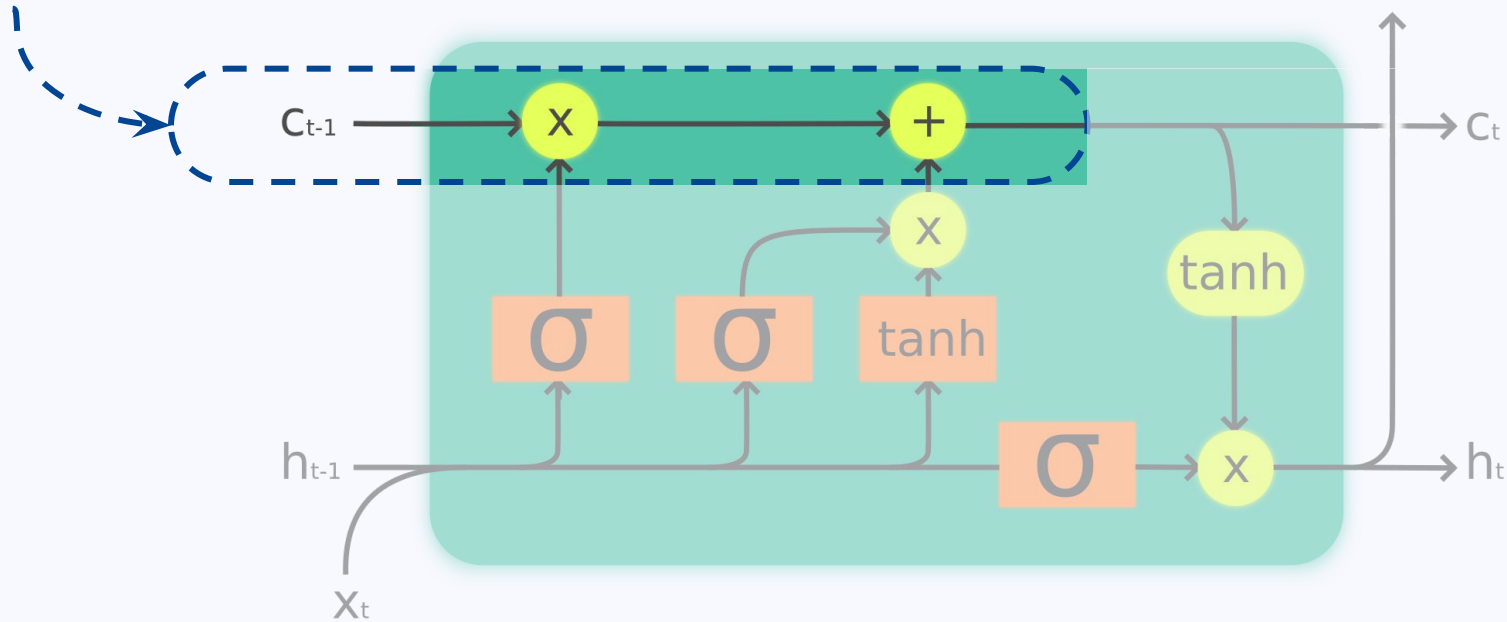
**Hidden State:** short-term memory, **updated** by weights and biases!





# LSTM: Cell

**Cell State:** long-term memory, **not affected** by weights and biases!

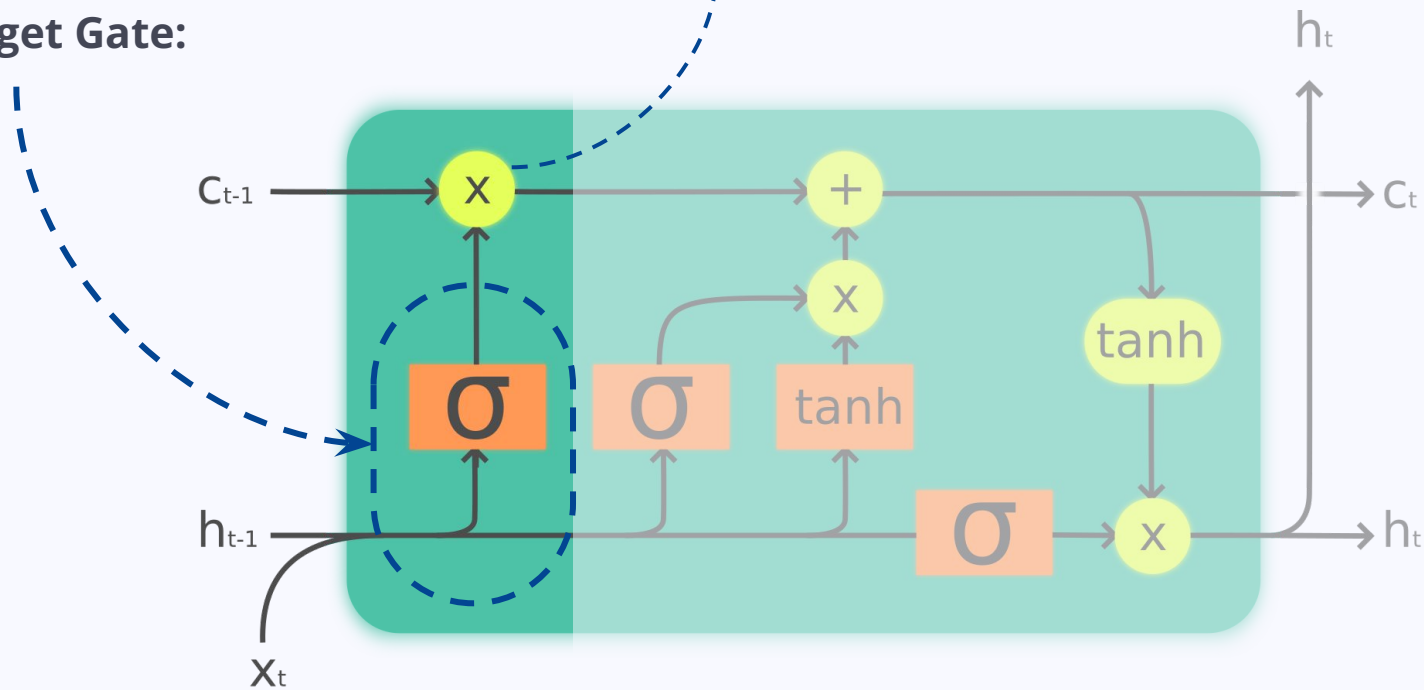


# LSTM: Cell

Forget Gate:

$\sigma$  = sigmoid activation function (0-1)

$x$  = % how much long-term memory to keep



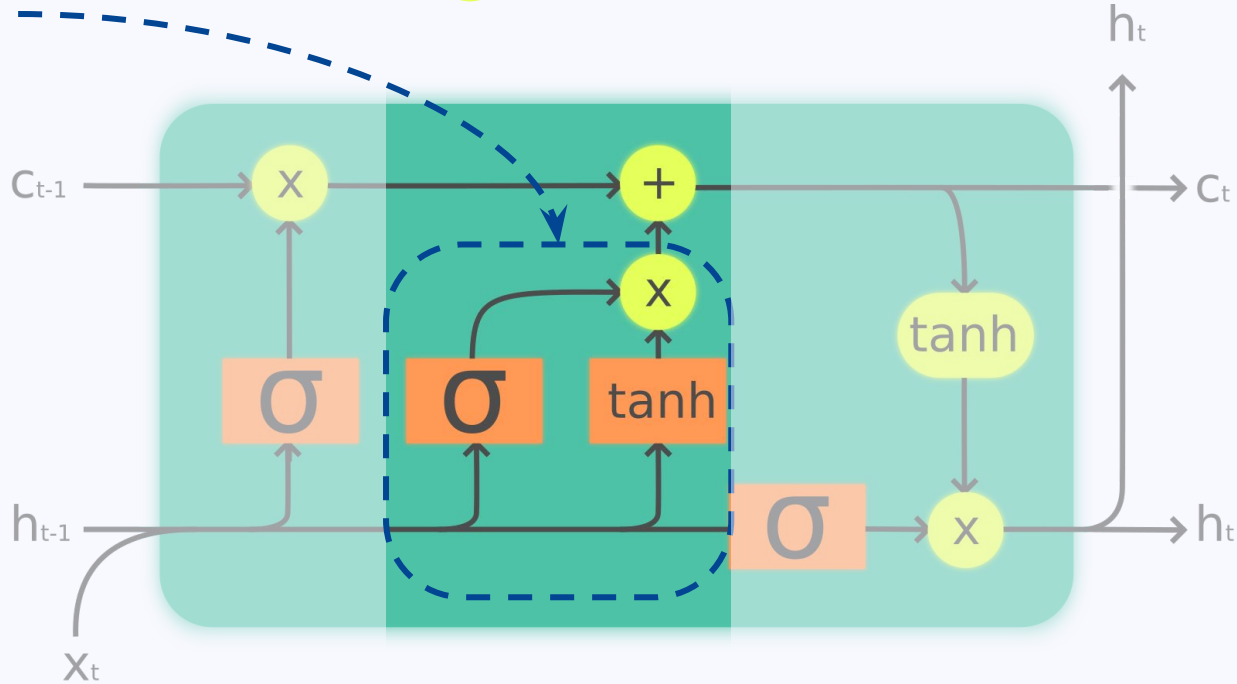
# LSTM: Cell

Input Gate:

$\tanh$  = tanh activation function (-1-1)

$\times$  = % of **new memory** to update long-term memory

$+$  = add to the existing **long-term memory**

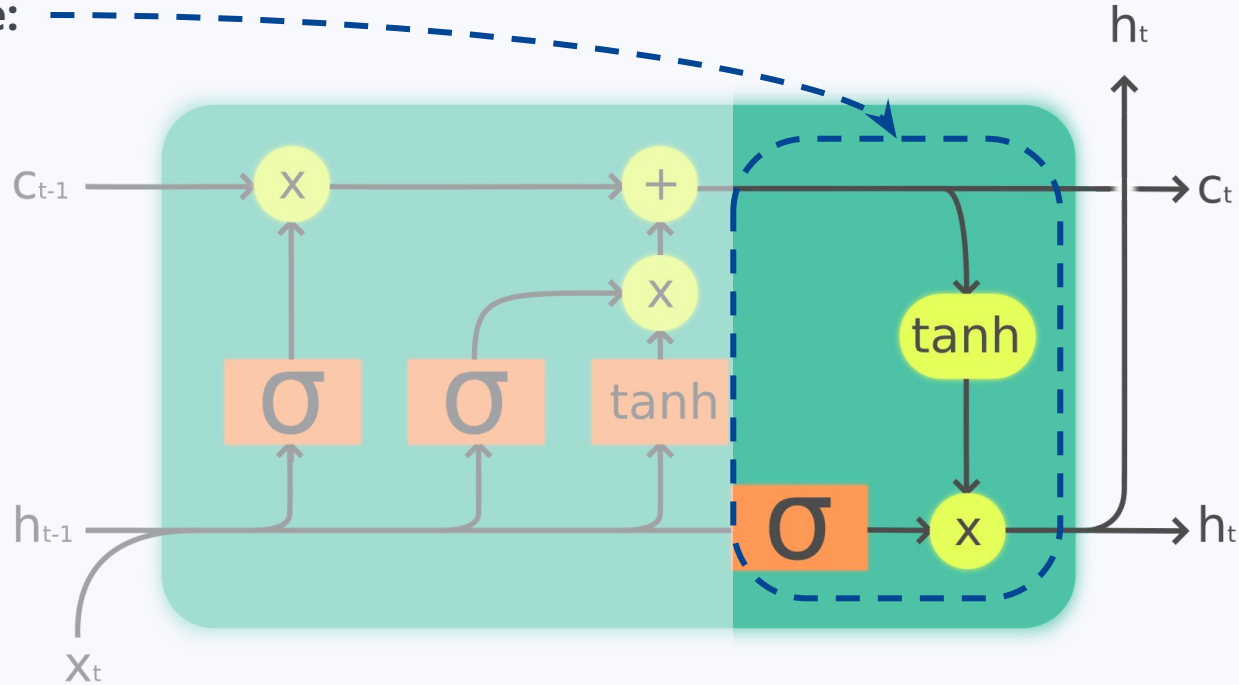


# LSTM: Cell

**tanh** = tanh with **new long-term memory**

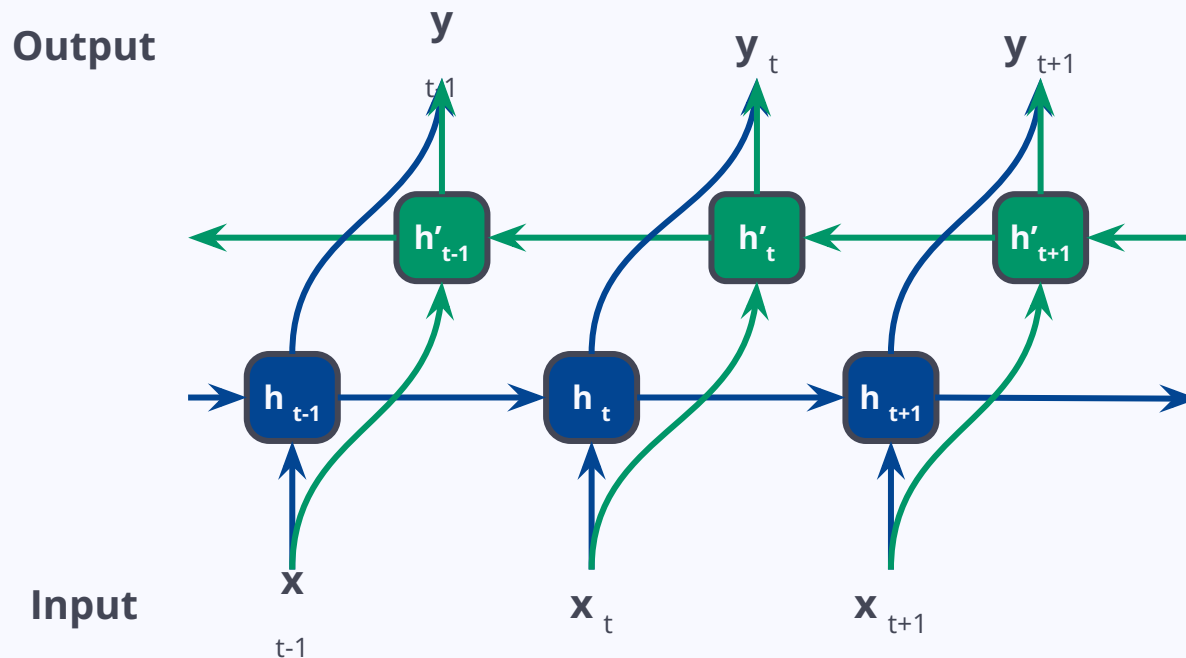
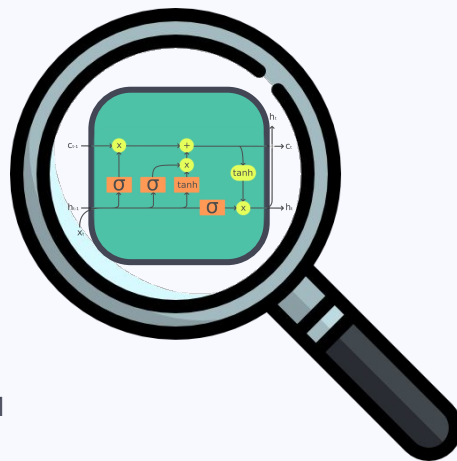
**x** = % of **new memory** to update short-term memory

Output Gate:



# LSTM: Model

e.g. a bidirectional LSTM!

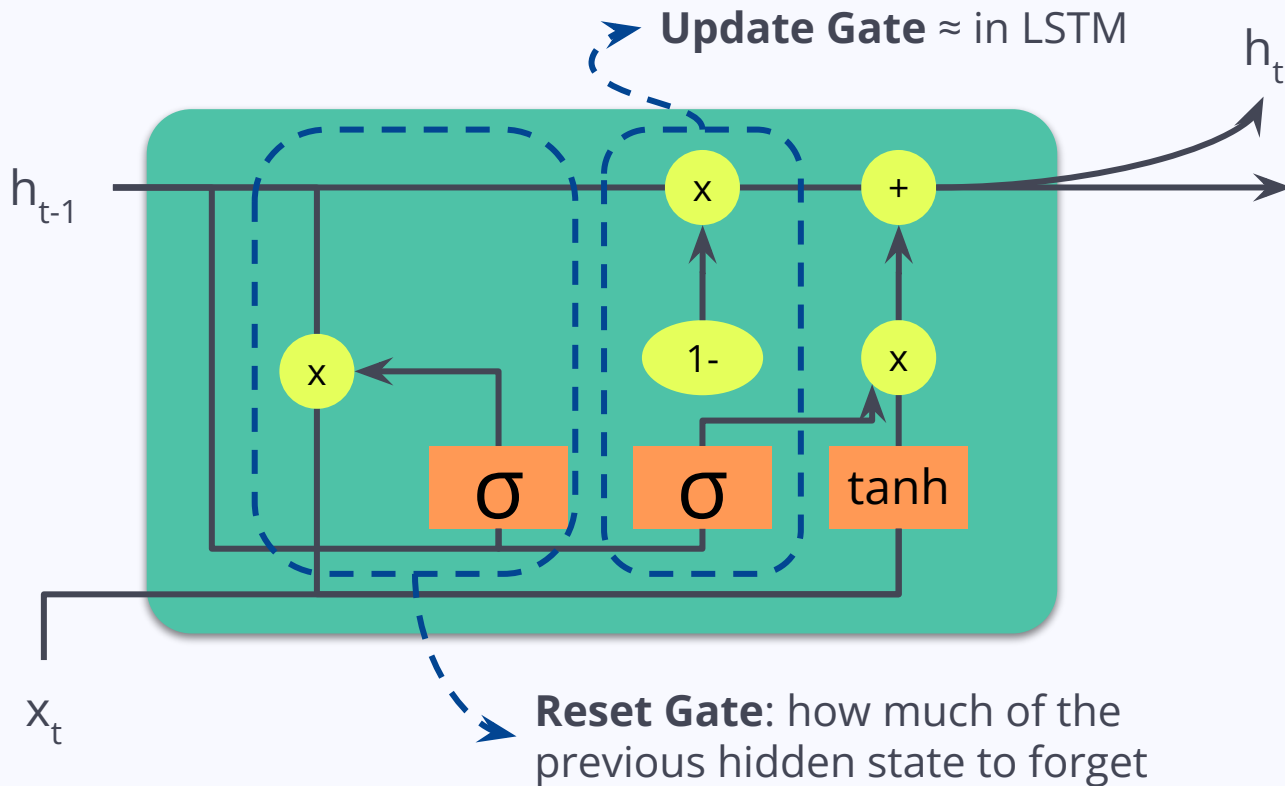


# LSTMs: Pros & Cons

- + **Memory unit** to learn long-term dependencies in sequential data
- + **Robust** to noisy data
- + **Flexible** for many tasks
- Slower to train than RNNs because of extra calculations
- Vanishing/exploding gradients can still be a problem...
- Prone to **overfitting**

# Gated Recurrent Units (GRU)

1- = inversion operation



# 4. Encoder-Decoder architecture

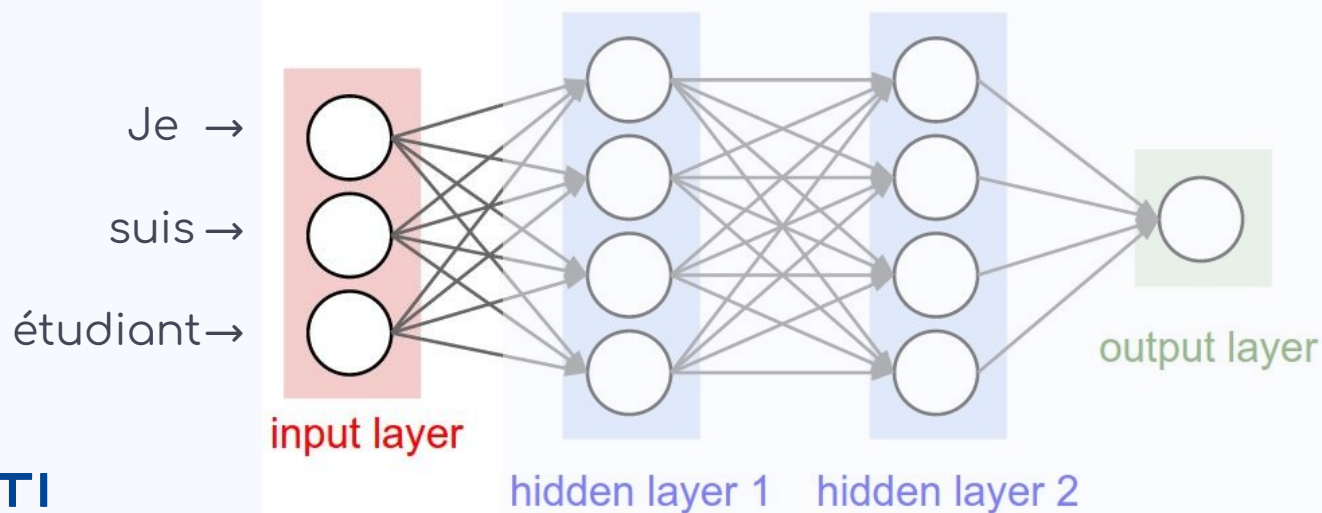


# What is an encoder?

- **Part** of a neural network architecture
- Made of feedforward NNs, RNNs, LSTMs, or GRUs, etc...
- Turns a sequence into numerical representations = **context vectors**
- Pure encoder model? Yes, e.g. the sentence classifier!

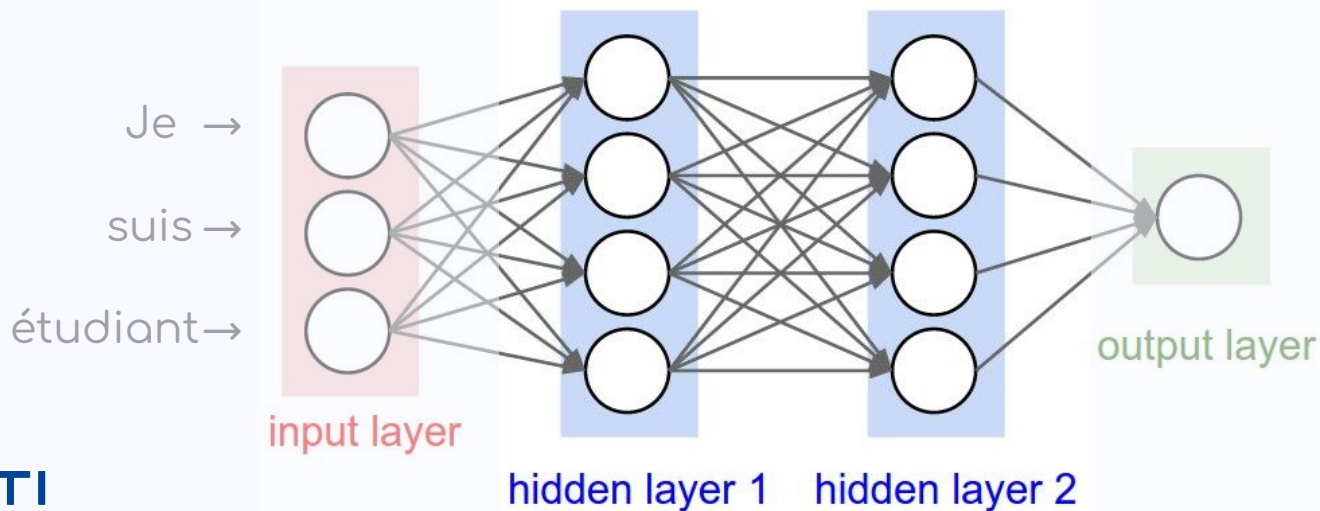
# What is an encoder?

1. **Encode** each input token



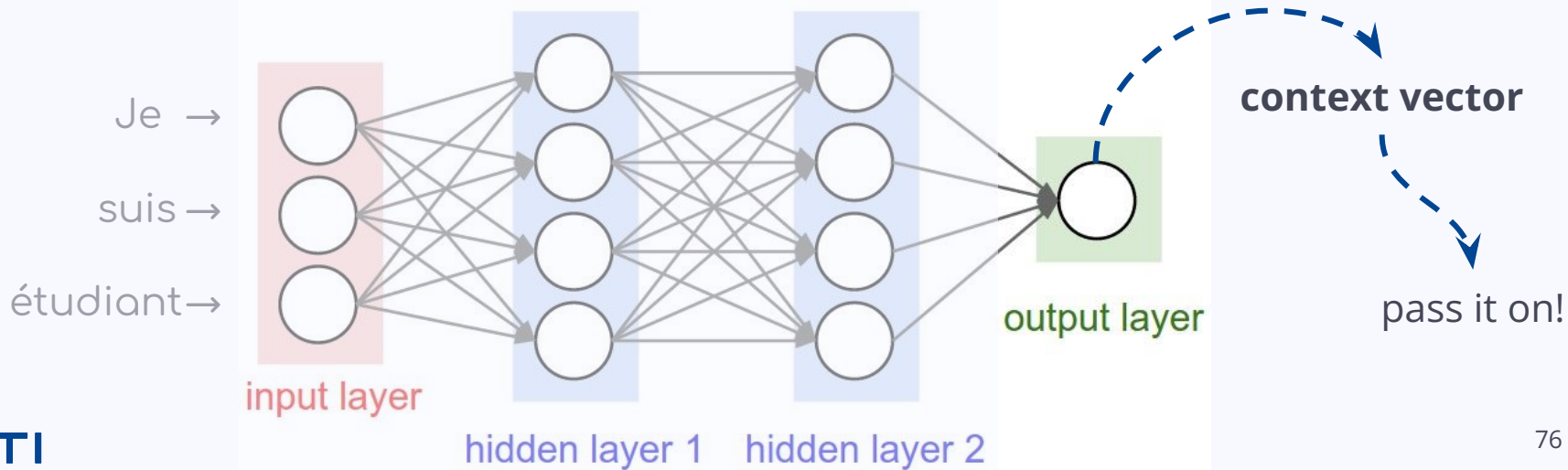
# What is an encoder?

1. **Encode** each input token
2. 🕒 → 🕒 → 🕒: **update** hidden state based on input + context (same w&b!)



# What is an encoder?

1. **Encode** each input token
2. 🕒 → 🕒 → 🕒: **update** hidden state based on input + context (same w&b!)
3. Encoder output = its final hidden state = **context vector**



# Encoder's context vector

- Remember word embeddings? e.g. word2vec
- **Representation of the input sequence**
- Reminder: we use these context vectors as a plug-in for other models!
- Spoiler: we can make some very powerful word embeddings with this information about a word's context...



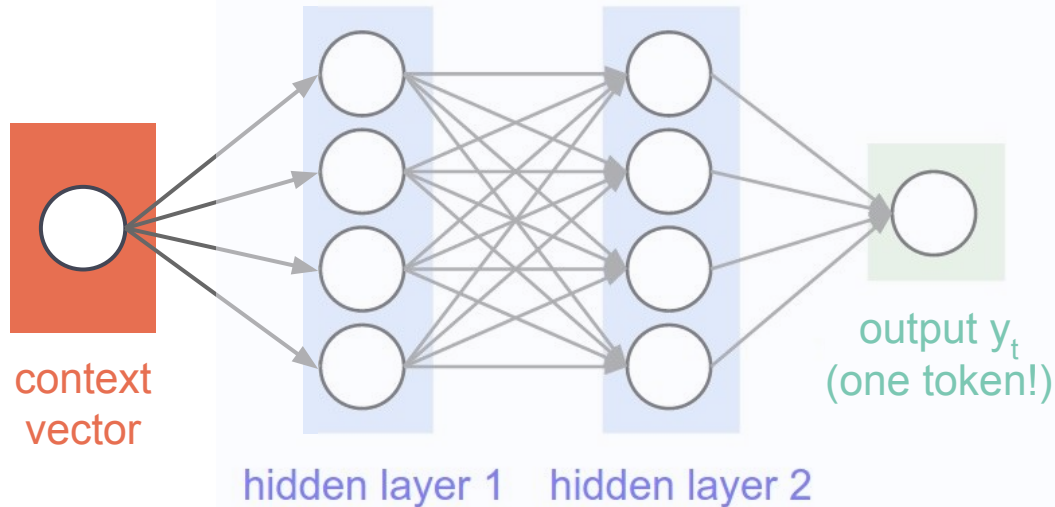
# What is a decoder?

- **Part** of a neural network architecture
- **Reconstruct the encoded representation**
- Work **one token** at a time by picking the token with the highest probability
- **Autoregressive generation**: predicting a token based on the previously generated tokens
- Pure decoder model? Yes!



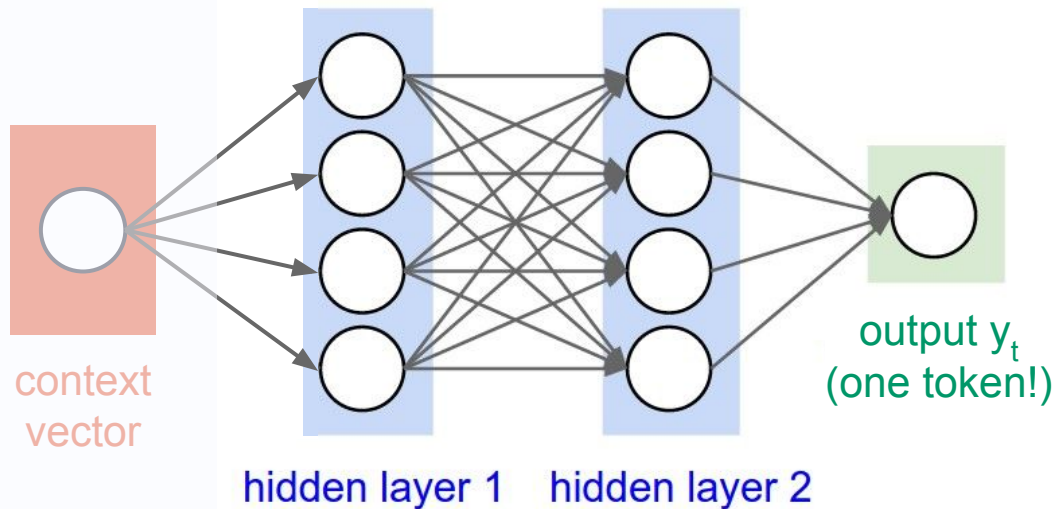
# What is a decoder?

1. **Input:** the context vector as initial hidden state



# What is a decoder?

1. **Input:** the context vector as initial hidden state
2. 🕒 → 🕒 → 🕒 : **predict** next token based on current hidden state + previously generated tokens

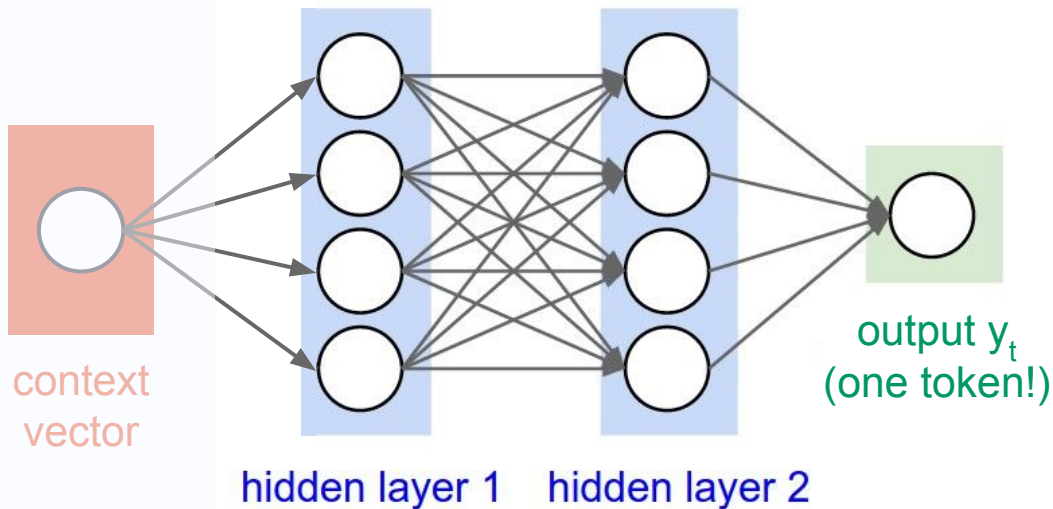




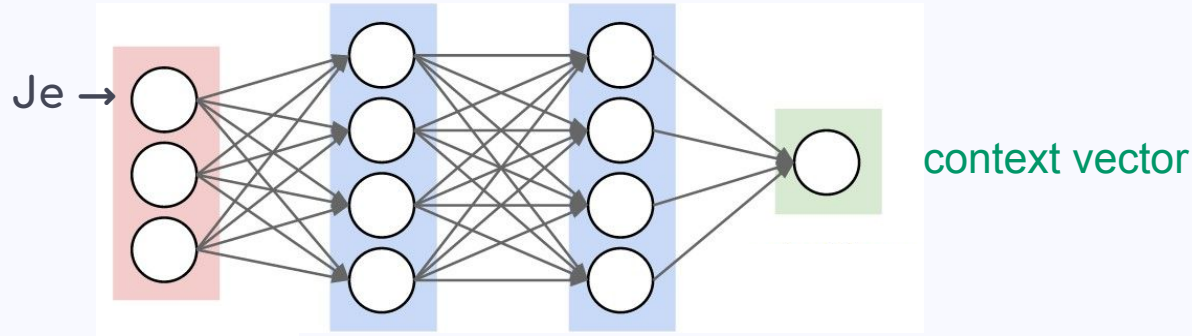
# What is a decoder?

1. **Input:** the context vector as initial hidden state
2. 🕒 → 🕒 → 🕒 : **predict** next token
3. **Stop** when it predicts <eos> or max length reached!

End  
of  
Sequence

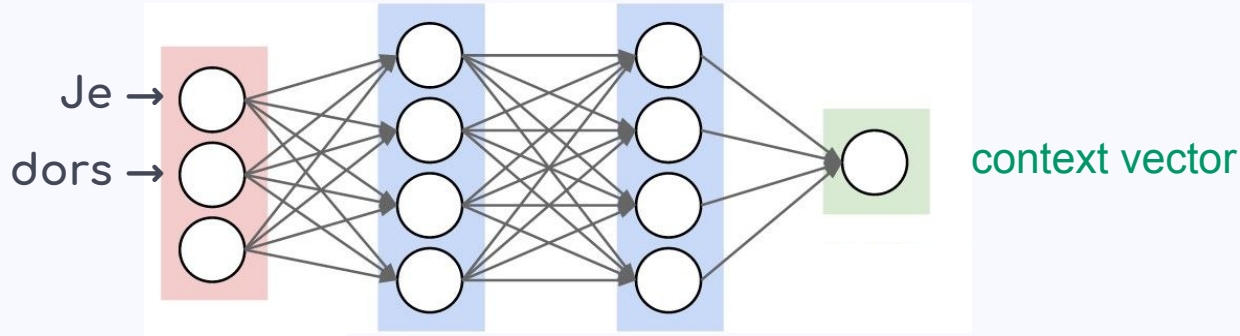


# Encoder-decoder architecture



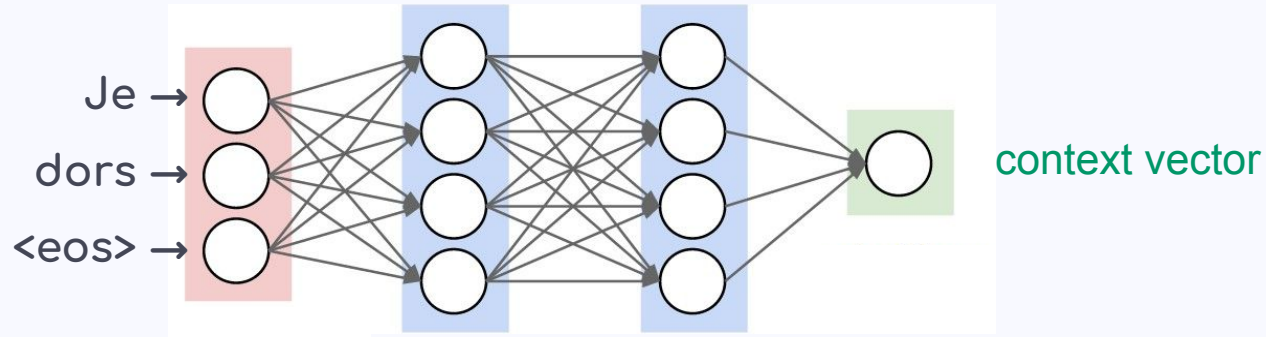
Je dors  
↓  
[Machine Translation]  
↓  
I am sleeping

# Encoder-decoder architecture



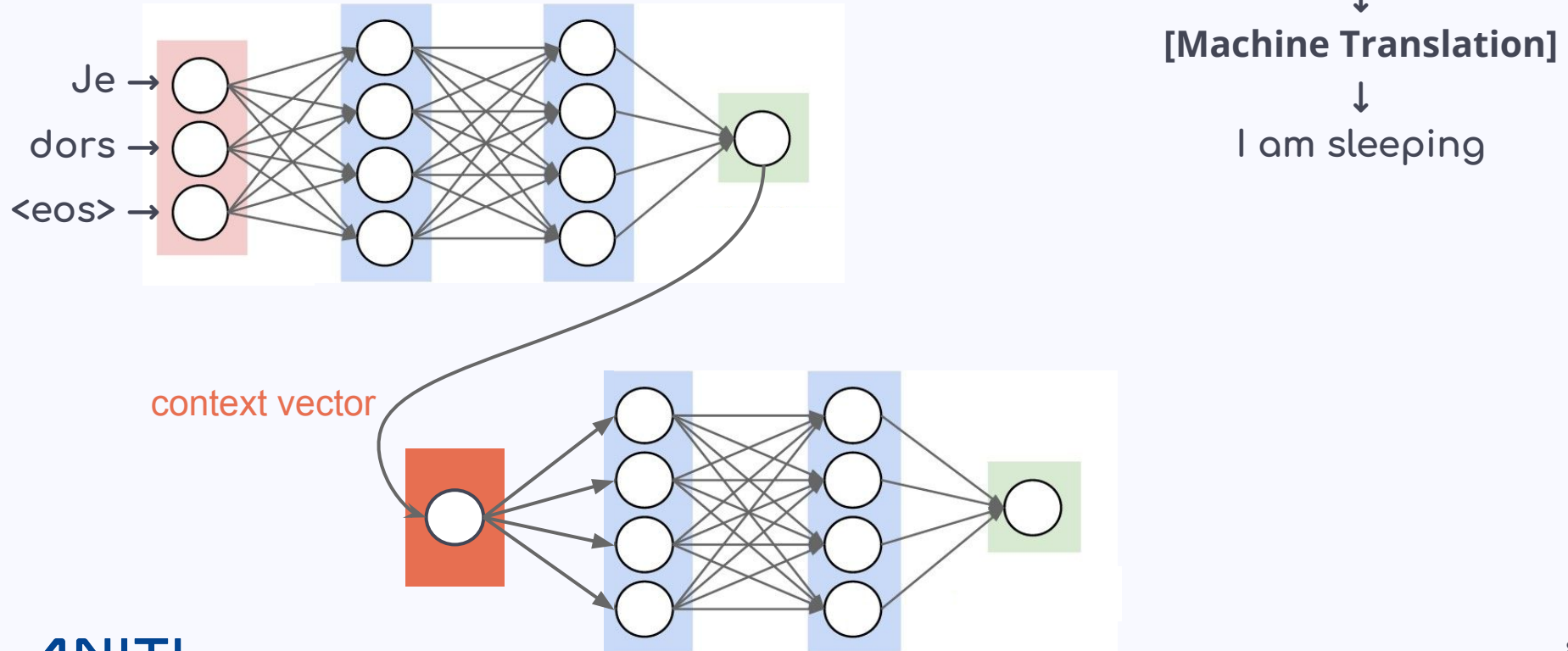
Je dors  
↓  
[Machine Translation]  
↓  
I am sleeping

# Encoder-decoder architecture

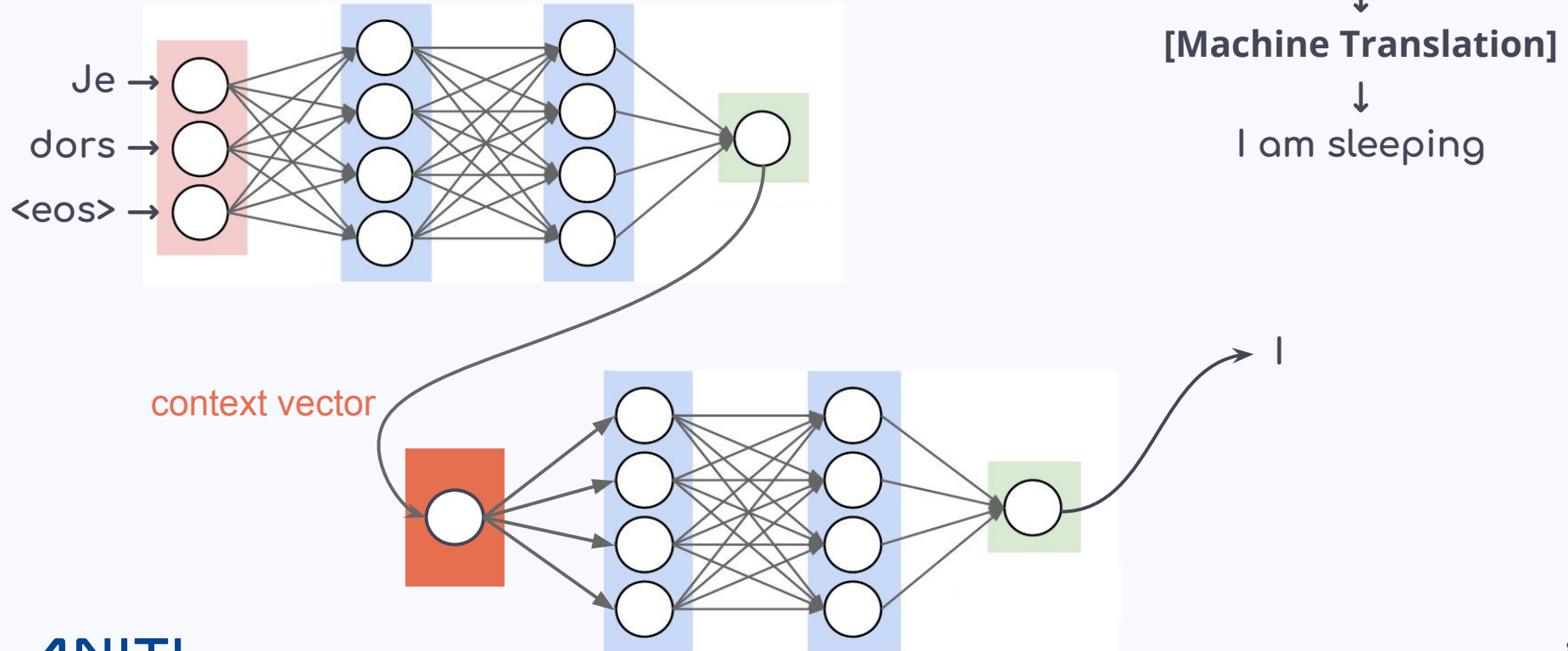


Je dors  
↓  
[Machine Translation]  
↓  
I am sleeping

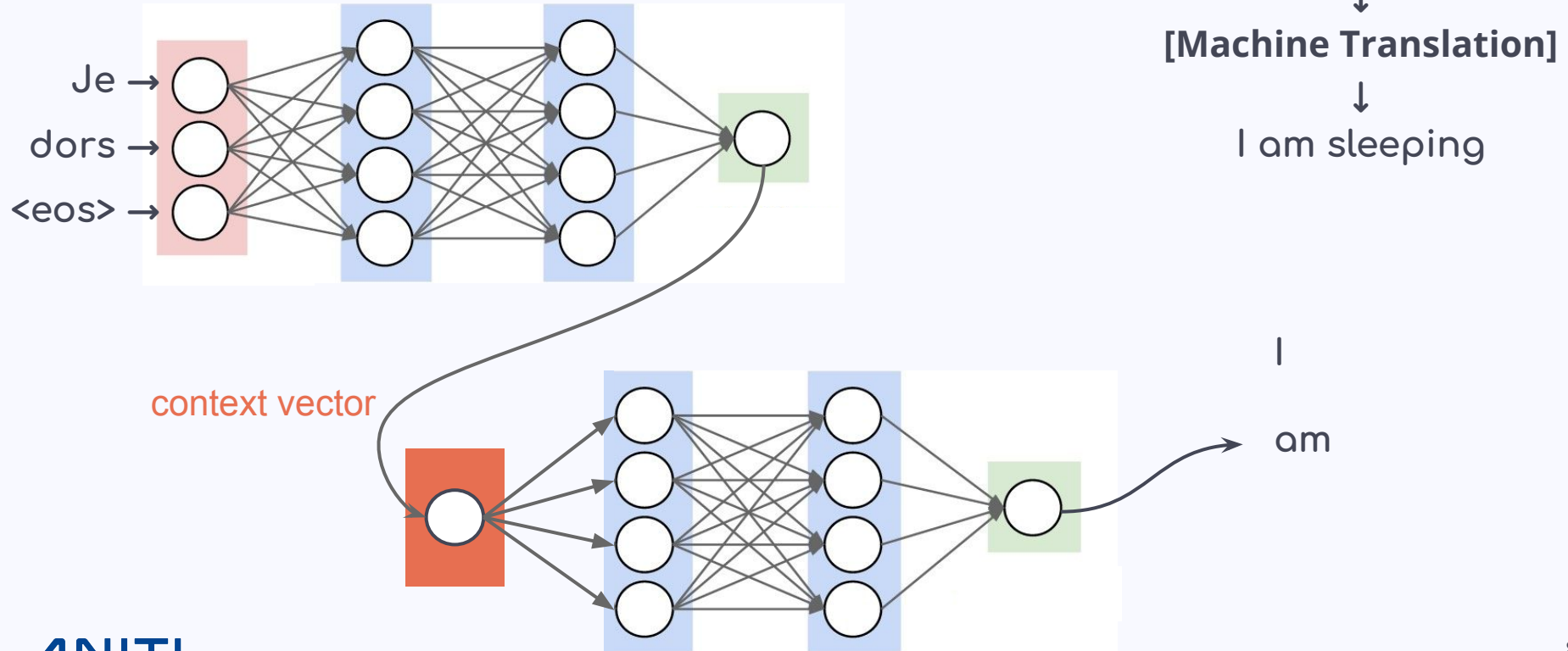
# Encoder-decoder architecture



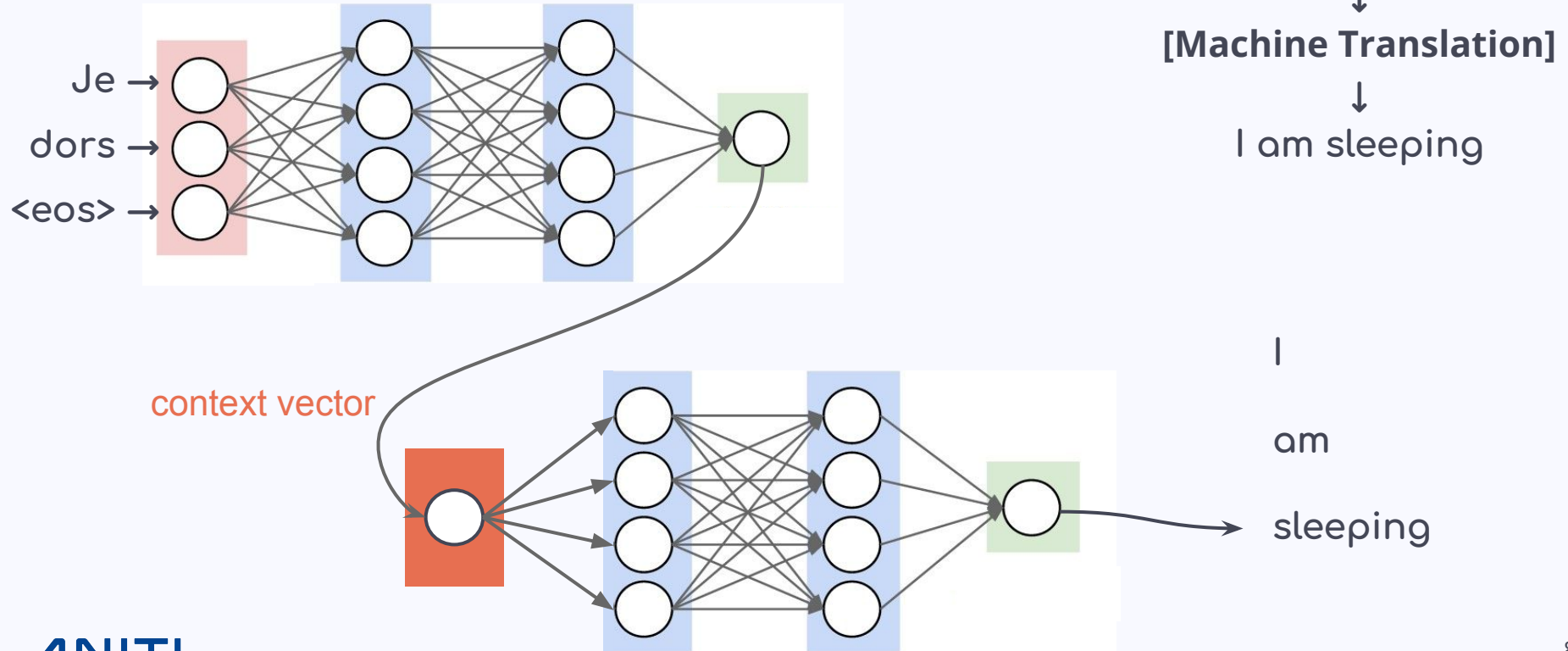
# Encoder-decoder architecture



# Encoder-decoder architecture

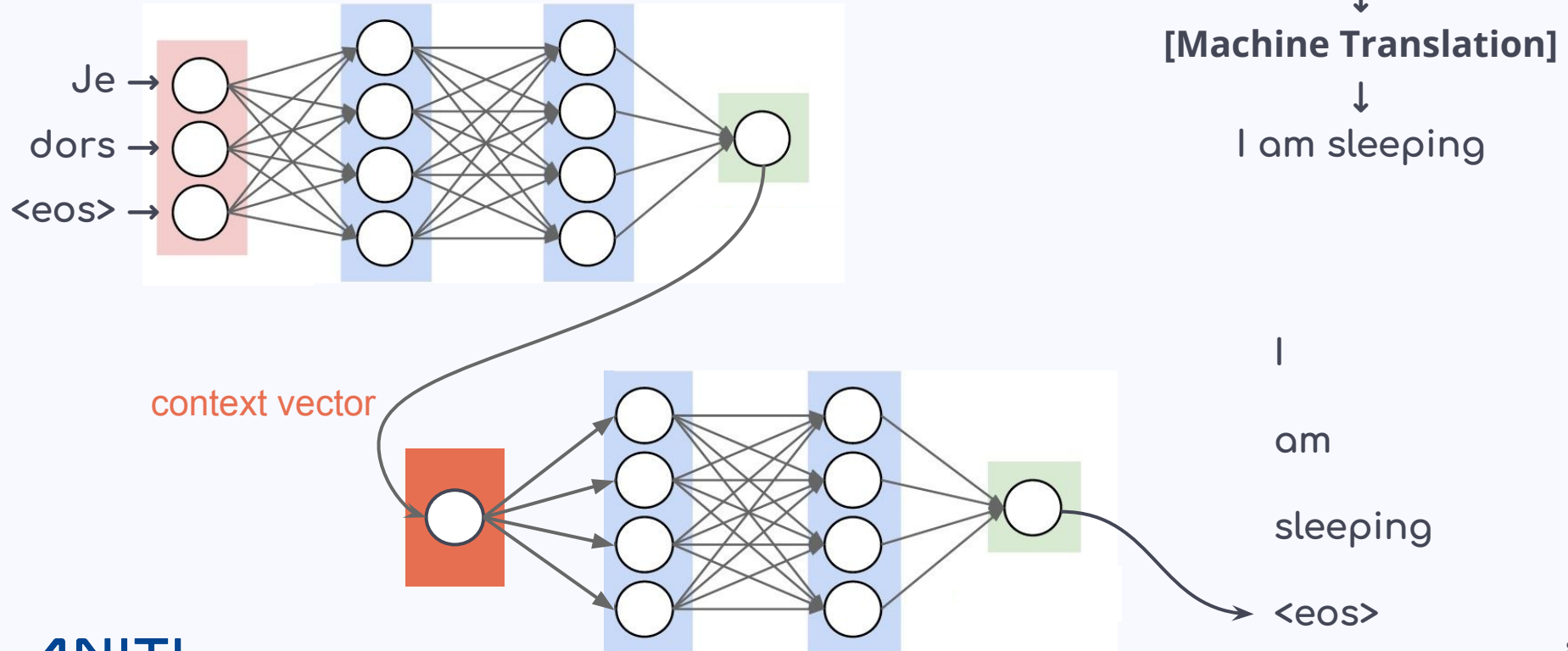


# Encoder-decoder architecture

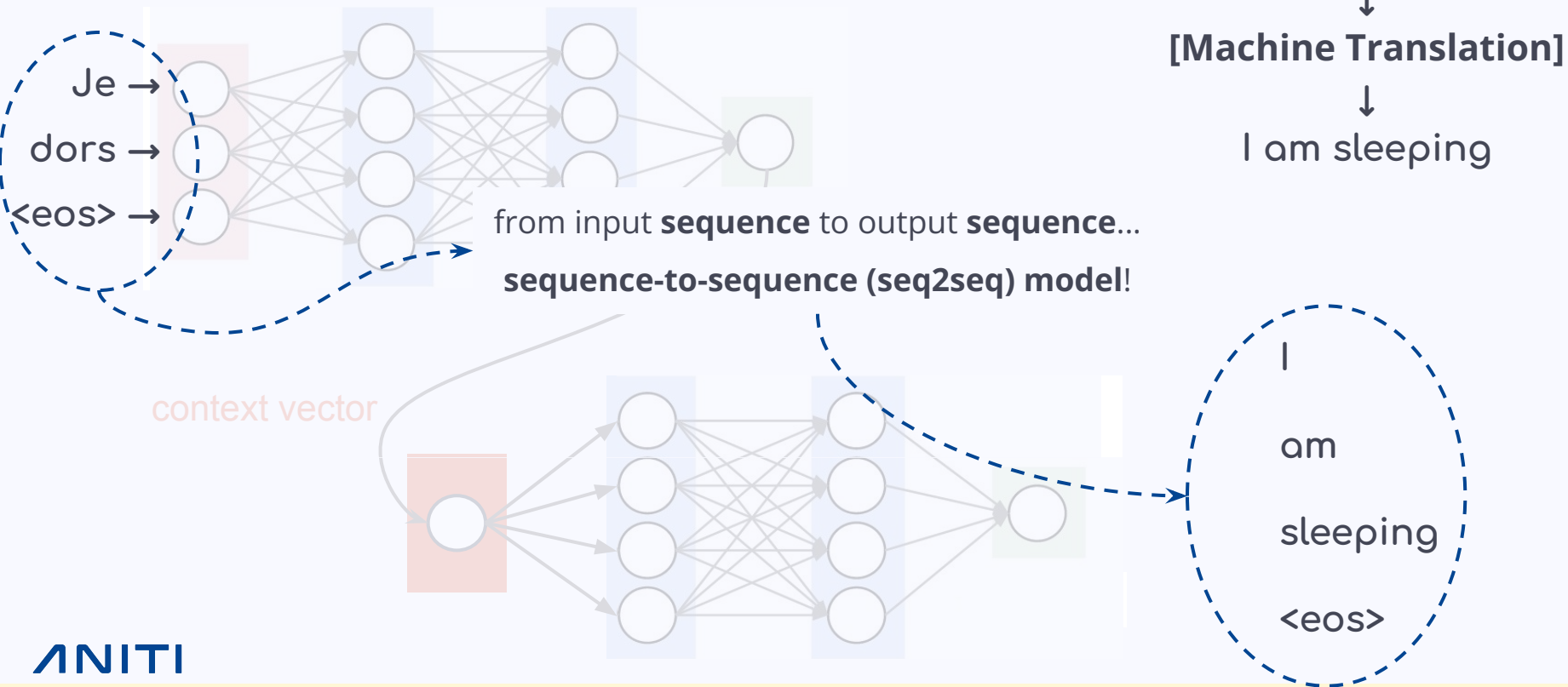




# Encoder-decoder architecture



# Encoder-decoder architecture



# Encoder-decoder: Pros & Cons

- + Better performance, adaptive
- + Generalizes across tasks
- + Adapted to NLP tasks with varying input/output lengths

# Encoder-decoder: Pros & Cons

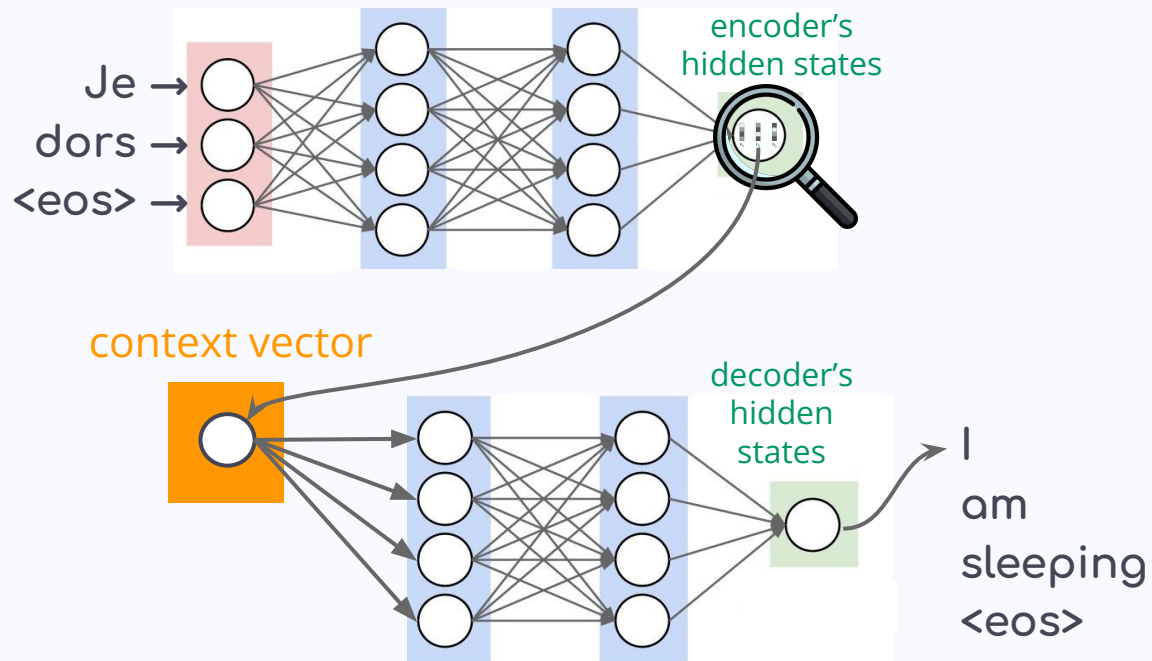
- + Better performance, adaptive
- + Generalizes across tasks
- + Adapted to NLP tasks with varying input/output lengths
- Lack of explicit alignment!
- Difficulty with long sequences!
- Over-reliance on context vector!

# 5. Attention mechanism

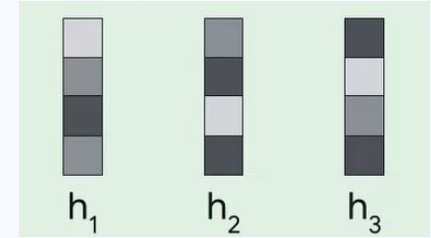
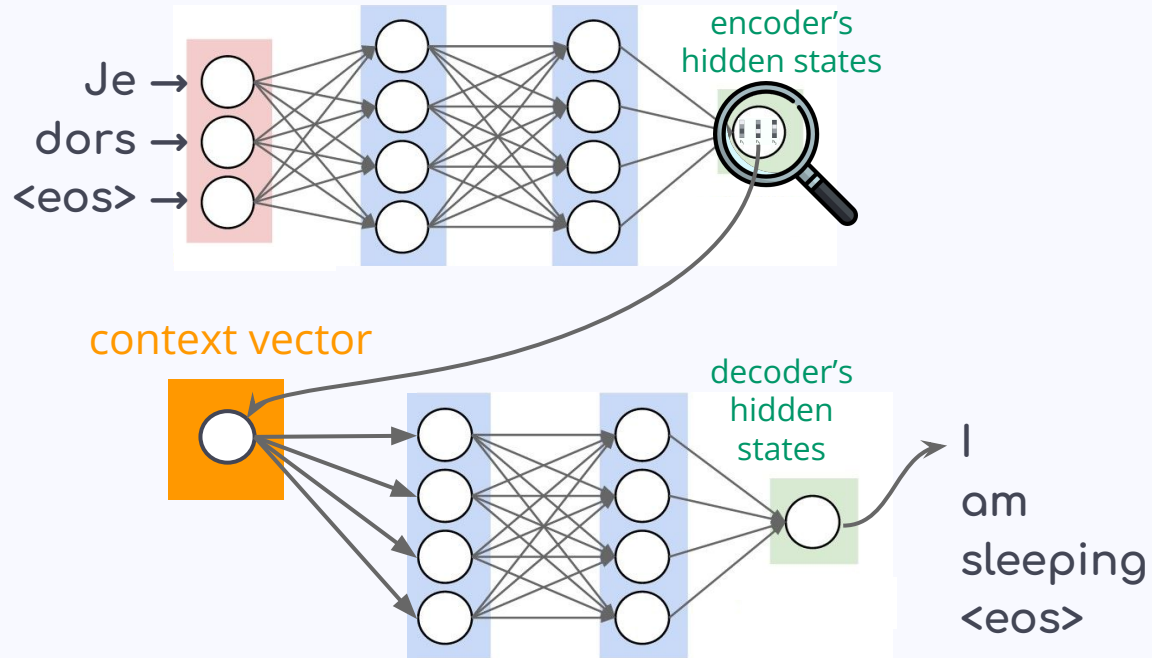
# What is Attention?

- A **similarity score** between each input token (after passing encoder) and each output token (after passing through decoder)
- **Attention score's** goal: highest scores between related tokens, e.g.  $Je \leftrightarrow I$
- **Attention mechanism**: a NN in-between Encoder-Decoder!

# Adding Attention to seq2seq

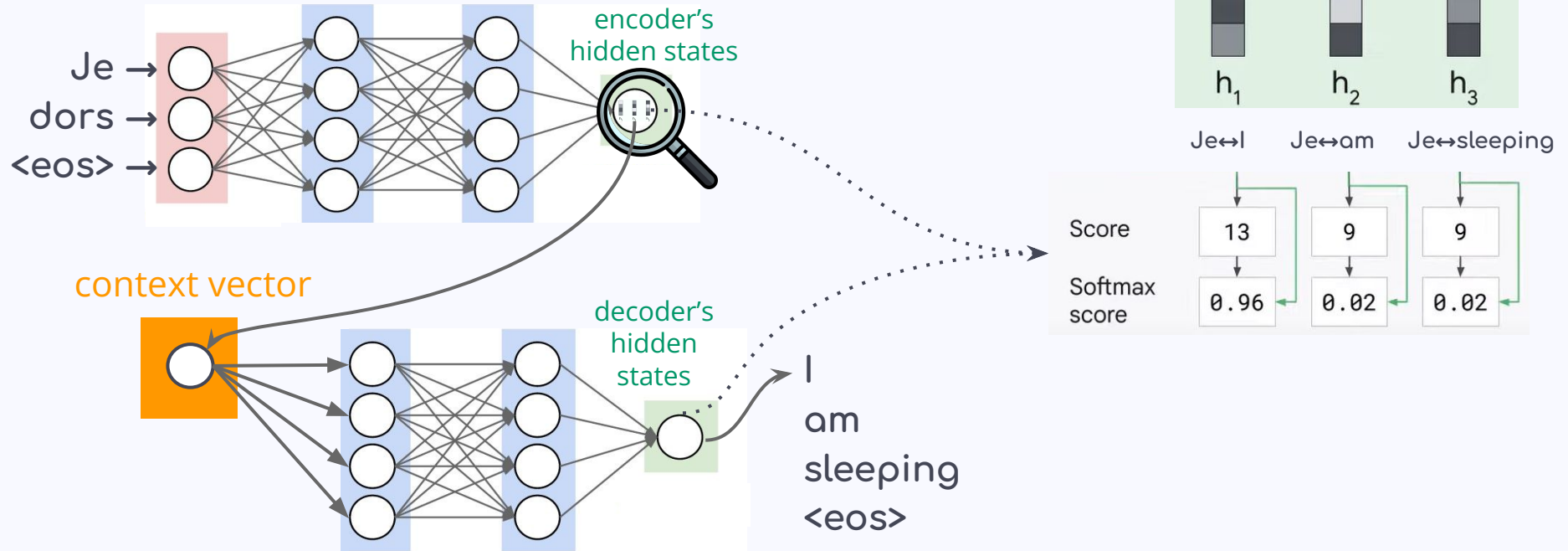


# Adding Attention to seq2seq

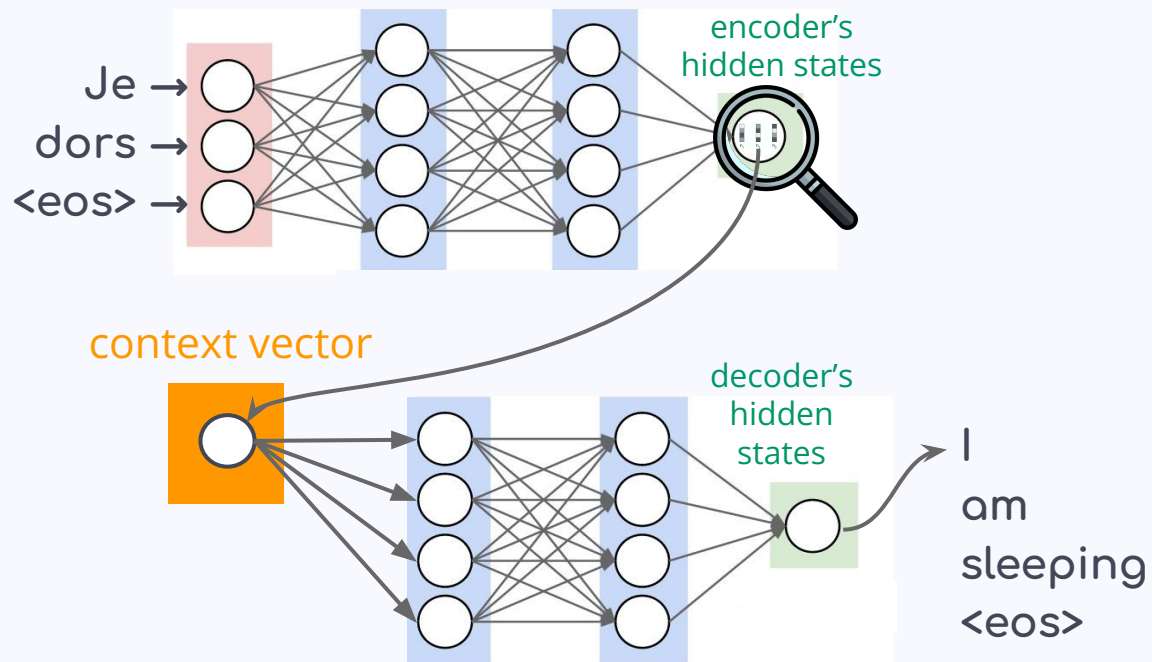




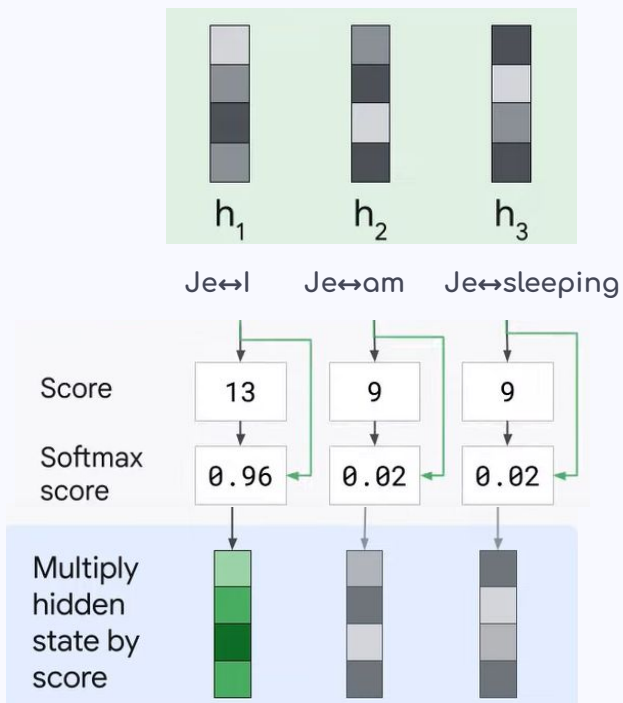
# Adding Attention to seq2seq



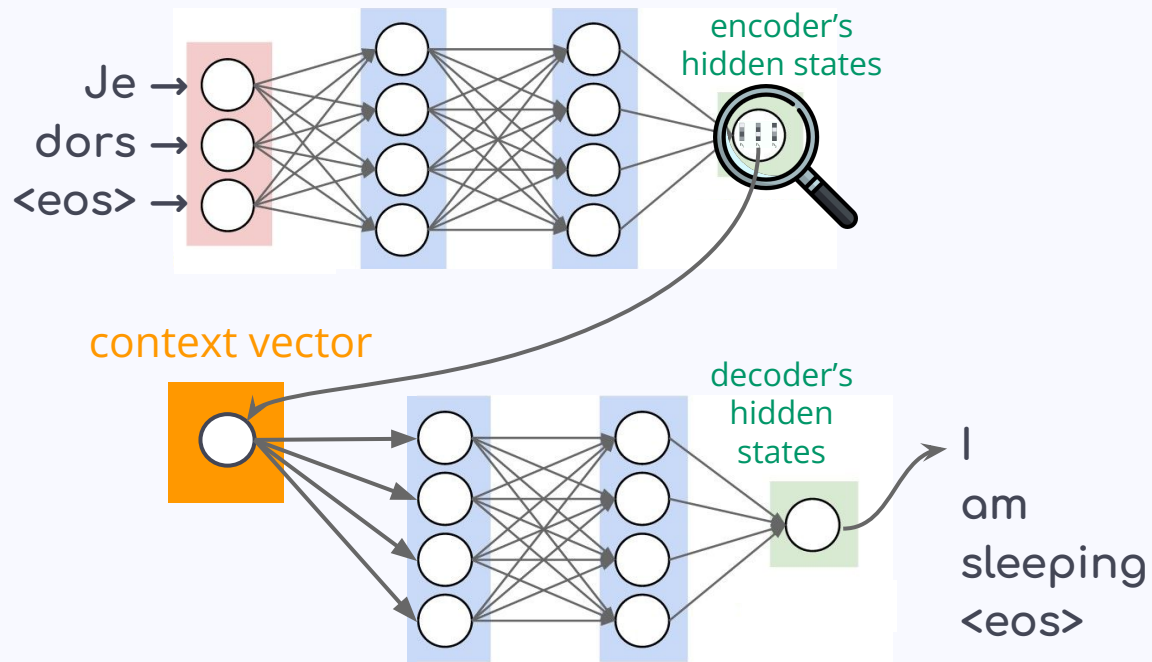
# Adding Attention to seq2seq



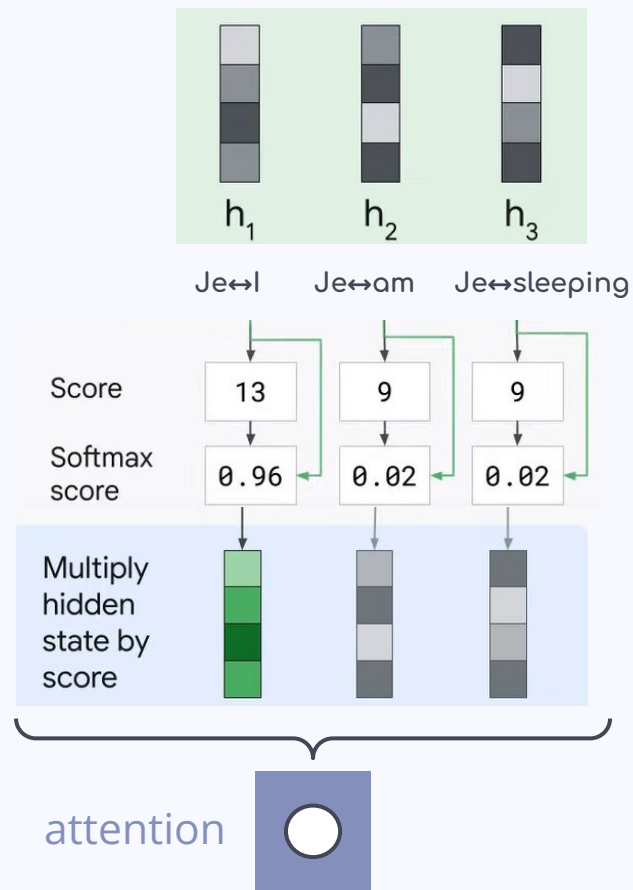
Inside the context vector...



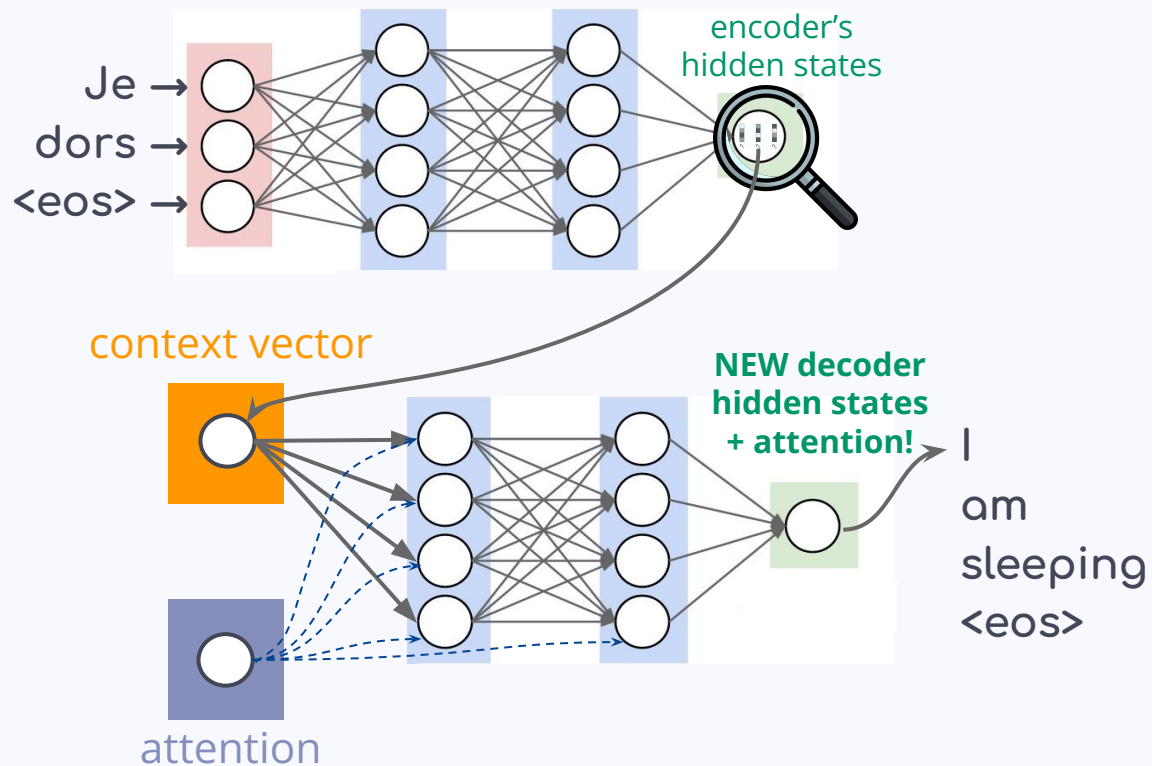
# Adding Attention to seq2seq



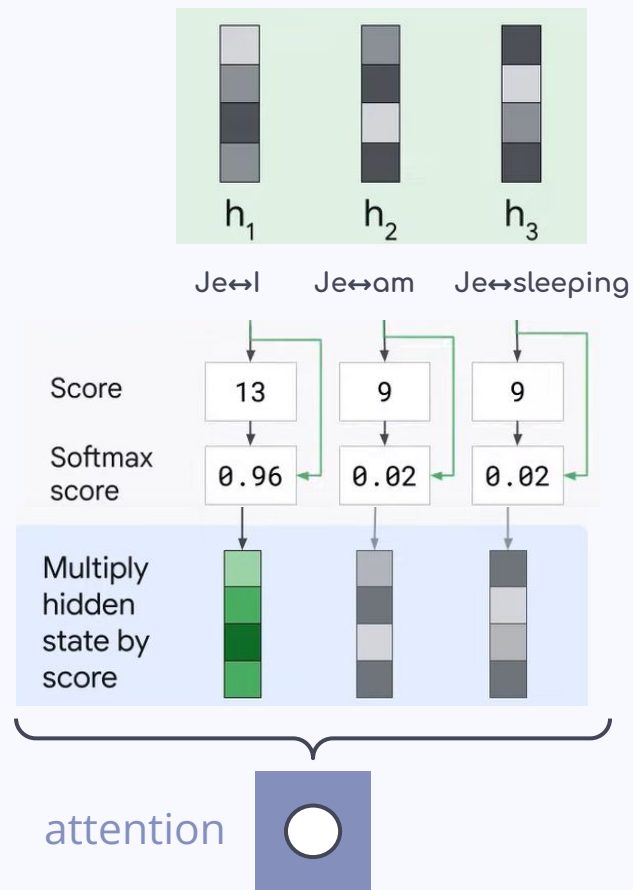
Inside the context vector...



# Adding Attention to seq2seq

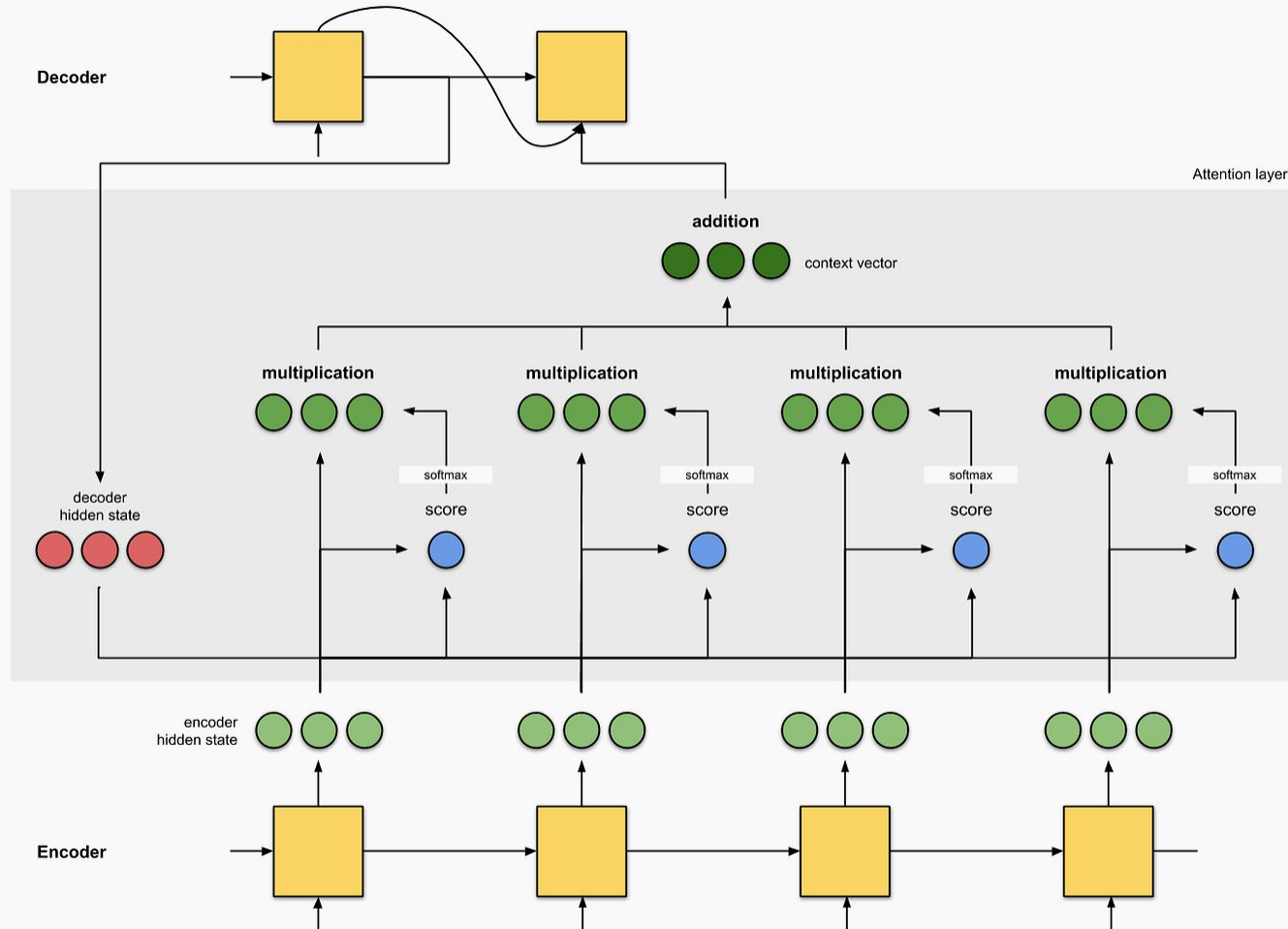


Inside the context vector...



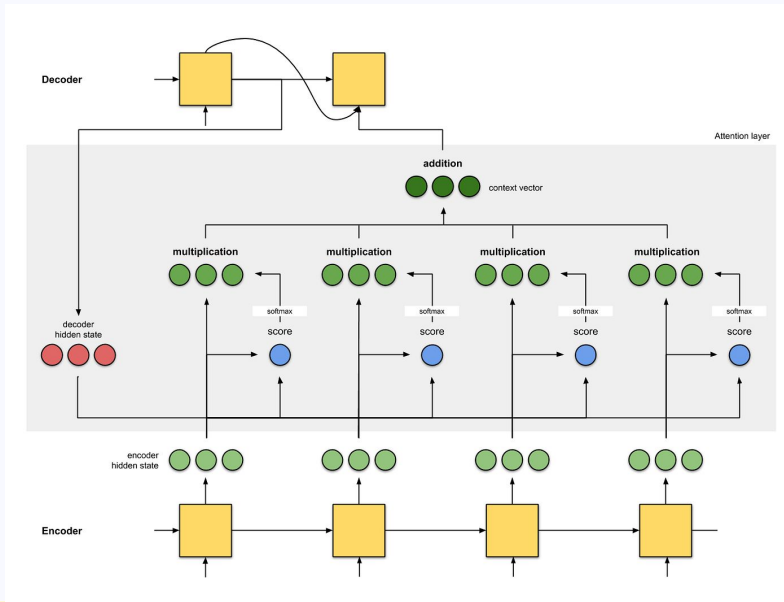
# The (ideal) attention weights





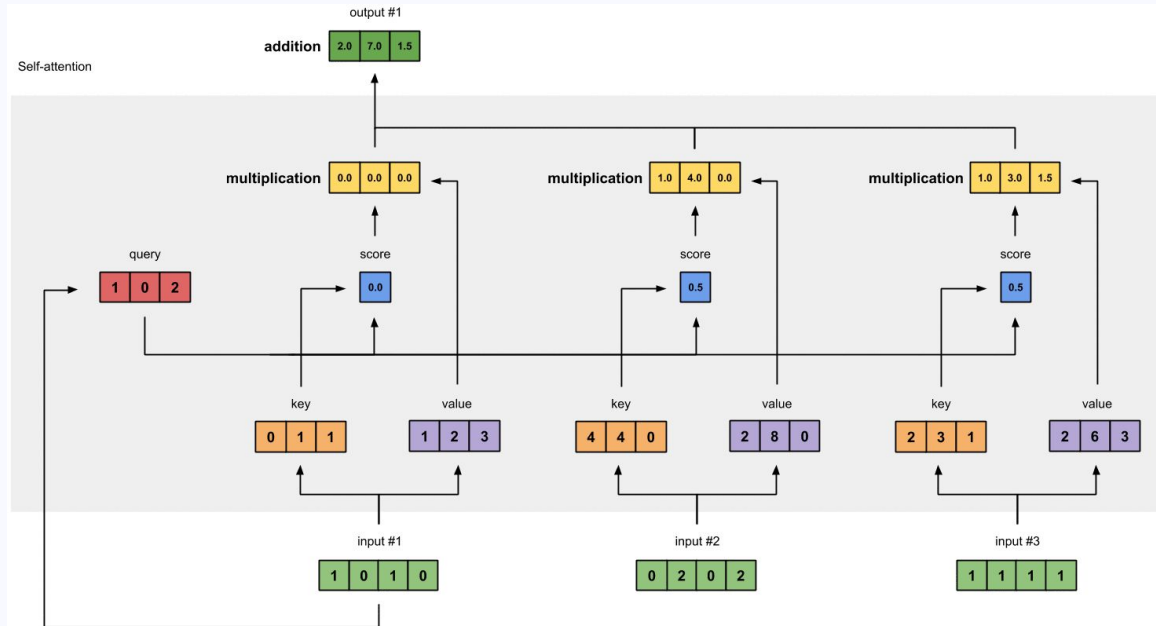
# But is it enough attention?

- Vaswani et al. (2017): **Self-attention** = foundation for transformers & LLMs!
- Goodbye seq2seq with attention...



# But is it enough attention?

- Vaswani et al. (2017): **Self-attention** = foundation for transformers & LLMs!
- Hello **transformer with self-attention**!





# Thank you for your attention (weights)!

Let's head to Google Colab!

<https://colab.research.google.com/drive/1MWqyE46KXhKfL-hpQBMhAHYSIsKejtal?usp=sharing>



# Practical session refresher

- **Index:** the number assigned to a token, e.g. 'ajout'  $\leftrightarrow$  23
- **Padding:** adding 0s to the sentence vector after <eos>, in order to make all our input sequences the same length (0 = ignored)
- **Dropout:** deactivate temporarily some nodes
- **Epoch:** one full training of the NN with all the data
  - **Forward/backward (pass):** one processing pass of the data in one direction
- **Batch:** how many sentences we will process simultaneously
- **Loss:** function that compares the target and predicted output values, aka how good/bad the predictions are!

# Further resources

- [A Neural Network Playground](#): visualize how neural networks look inside!
- [Google tutorials on Neural networks](#) ([en français](#))
-  (Reliable) reading material:
  - [The Unreasonable Effectiveness of Recurrent Neural Networks](#)
  - [Visualizing A Neural Machine Translation Model](#) - [en français](#)
  - [Dive into Deep Learning](#) - everything NN: book, code, math (advanced!)
-  YouTube tutorials:
  - [3Blue1Brown](#)
  - [StatQuest](#)
  - [The AI hacker](#)